Qudit-based scalable quantum algorithm for solving the integer programming problem

Kapil Goswami,[1,*] Peter Schmelcher,[1,2,†] and Rick Mukherjee[1,3,4,‡]

[1] *Zentrum für Optische Quantentechnologien, Universität Hamburg, Luruper Chaussee 149, 22761 Hamburg, Germany*
[2] *The Hamburg Centre for Ultrafast Imaging, Universität Hamburg, Luruper Chaussee 149, 22761 Hamburg, Germany*
[3] *Department of Physics & Astronomy, University of Tennessee, Chattanooga, TN 37403, USA*
[4] *UTC Quantum Center, University of Tennessee, Chattanooga, TN 37403, USA*

(Dated: August 20, 2025)

Integer programming (IP) is an NP-hard combinatorial optimization problem that is widely used to represent a diverse set of real-world problems spanning multiple fields, such as finance, engineering, logistics, and operations research. It is a hard problem to solve using classical algorithms, as its complexity increases exponentially with problem size. Most quantum algorithms for solving IP are highly resource inefficient because they encode integers into qubits. In [1], the issue of resource inefficiency was addressed by mapping integer variables to qudits. However, [1] has limited practical value due to a lack of scalability to multiple qudits to encode larger problems. In this work, by extending upon the ideas of [1], a circuit-based scalable quantum algorithm is presented using multiple interacting qudits for which we show a quantum speed-up. The quantum algorithm consists of a *distillation function* that efficiently separates the feasible from the infeasible regions, a phase-amplitude encoding for the cost function, and a quantum phase estimation coupled with a multi-controlled single-qubit rotation for optimization. We prove that the optimal solution has the maximum probability of being measured in our algorithm. The time complexity for the quantum algorithm is shown to be $O(d^{n/2} + m \cdot n^2 \cdot \log d + n/\epsilon_{QPE})$ for a problem with the number of variables n taking d integer values, satisfying m constraints with a precision of ϵ_{QPE}. Compared to the classical time complexity of brute force $O(d^n)$ and the best classical exact algorithm $O((\log n)^{3n})$, it incurs a reduction of $d^{n/2}$ in the time complexity in terms of n for solving a general polynomial IP problem.

arXiv:2508.13906v1 [quant-ph] 19 Aug 2025

One pertinent question while building new quantum algorithms is whether they can provide a quantum advantage over their classical counterparts [2–5]. Another related question is to look for problems for which the quantum algorithms can play a role in solving the NP-hard problems efficiently [6, 7]. In this work, both of these questions are addressed for a well-known combinatorial optimization problem with widespread real-world applications: the integer programming (IP) problem [8, 9].

Quantum advantage between algorithms can be assessed by comparing the number of queries made to an *oracle* by each algorithm to find the solution. An oracle is a "black-box" subroutine that computes a function f in a single step, with access being limited to the input and output only. This led to the development of many oracle-based quantum algorithms, such as the Deutsch-Jozsa (DJ), Bernstein-Vazirani (BV), and Simon's algorithms [10–14], where quantum advantage can be explicitly evaluated compared to the classical algorithms. Since practical realization of oracles using quantum circuits can be difficult, the gate-based approaches provide alternatives for the oracle subroutines, some of them are Shor's algorithm, the Quantum Fourier Transform (QFT), quantum phase estimation (QPE), and Grover's algorithm [15–18]. Although implementing oracles with gates can be cumbersome and expensive, nevertheless, these gate-based approaches set tangible targets required for quantum advantage, such as the number of qubits, the types of gates, the number of gates, and the acceptable error rates, all of which contribute to the threshold arguments [19].

We employ a similar approach to develop a quantum algorithm for integer programming, specifically utilizing a gate-based algorithm that incorporates qudits. Recently, there have been many developments in quantum computing using qudits instead of qubits [20–26]. Qudits are d-dimensional objects [20] that provide a larger Hilbert space (d^n for n d-dimensional qudits) to explore and store information. This helps to reduce circuit complexity and enhances the efficiency of the computations performed [20]. In [1], small IP problems were solved using a single *qudit*, which was a significant resource advantage compared to other quantum algorithms for IP. It, however, lacked two crucial aspects: an explicit quantum advantage and scalability to larger problem sizes. In this work, a scalable quantum algorithm is proposed that utilizes qudits to solve IP with an exponential reduction in time complexity compared to classical algorithms.

We propose to use a qudit array to store the integer variables, and a qubit array keeps track of the constraint satisfiability; both of the arrays are entangled in the circuit. Solving a given IP problem is divided into two tasks, similar to [1]: satisfying the constraints and optimizing the cost function. The constraints are satisfied by defining a *distillation function* that separates the feasible from the infeasible region; the qudit array is made to collapse onto the subspace where constraints of the IP problem are satisfied by measuring the qubit array in a specific state. This is followed by applying well-established methods such as quantum phase estimation and multi-controlled rotation, and the probability of the optimal solution is maximized.

Apart from resource efficiency over qubits, our proposed algorithm shows reduced time complexity over classical algorithms. For an IP problem defined with n integers taking d distinct values and m constraints, the time for the quantum al-

* kapil.goswami@uni-hamburg.de
† peter.schmelcher@uni-hamburg.de
‡ rick-mukherjee@utc.edu

gorithm scales as $T_Q \sim O(d^{n/2} + m \cdot n^2 \cdot \log d + n/\epsilon_{QPE})$ (in the worst case; only one state in the feasible region) to solve the problem within ϵ_{QPE} precision. This is a significant reduction of $d^{n/2}$ from the classical brute force time complexity of $T_{Cl} \sim O(d^n)$, which is the best-known time complexity scaling for solving a general polynomial-IP, especially for non-linear problems for which the best classical approximate algorithms also fail. Along with significantly reduced time complexity, another consequence of our algorithm is that it always ensures the optimal configuration in the qudit-qubit space.

Here is the outline of the manuscript. In Section I, we briefly define a general IP problem. Section II covers the main techniques that are used to construct the quantum algorithm. In Section III, we showcase the numerical implementation of the quantum algorithm along with the comparison with classical algorithms. The complexity analysis and the success probability for the algorithm are described in Section IV. We provide our conclusions in Section V, where we also make comments about the robustness of the algorithm on real noisy hardware.

I. PROBLEM DEFINITION

The classical algorithms for many optimization problems are reaching their limits nowadays, presenting both new opportunities and challenges for quantum algorithms [6, 7]. The optimization problems often fall into the NP-hard complexity class, making it exponentially difficult to solve using classical algorithms as the problem size increases [27–29]. It is widely believed that quantum algorithms, much like their classical counterparts, cannot solve NP-hard problems in polynomial time [30, 31]. However, they may still offer significant advantages, such as a polynomial speed-up or an exponential reduction in complexity (which reduces the base of the exponent), resulting in a huge practical advantage [18, 32, 33]. There are, broadly speaking, two mathematical structures that describe these optimization problems, namely quadratic unconstrained binary optimization (QUBO) and integer Programming (IP) [8, 9]. In the context of quantum algorithms, QUBO is a preferred choice due to its natural mapping to spin Hamiltonians [34–41]. For classical algorithms, most optimization problems are solved using integer programming (IP) or, in the general case, mixed-integer programming (MIP) rather than QUBO [28, 42, 43]. This is because IP offers a more general mathematical structure for encoding real-world problems, which often contain discrete variables and multiple constraints [8, 29]. The quantum algorithms for solving IP are highly resource-inefficient due to the lack of a direct mapping to encode discrete-integer variables onto a quantum system; instead, they rely on mapping the integer variables to binary [44–46]. One demonstration of mapping an optimization problem directly to a quantum system is presented in [47] that utilizes a geometric interpretation of a qubit.

For a general bounded variables IP problem with n decision variables each taking d values, the goal is to maximize the polynomial-cost function $C(\mathbf{x})$ represented as

$$C(\mathbf{x}) = c_1 x_1 \cdots x_{n'} + \cdots + c_v x_1 \cdots x_{n''}, \tag{1}$$

where $n', n'' \in \mathbb{Z}^+ \leq n$, v is the number of monomial terms in the cost function, c_j are the corresponding real-valued coefficients, and $x_i \in \{0, ..., d-1\}$. There are m constraints for the problem given as

$$\begin{aligned} C_1 &= a_{11} x_{q_1} \cdots x_{q_{k_1}} + \cdots \leq h_1, \\ &\vdots \\ C_m &= a_{m1} x_{r_1} \cdots x_{r_{k_m}} + \cdots \geq h_m, \end{aligned} \tag{2}$$

where $q_j, r_j \in \mathbb{Z}^+$, a_{uv} are the real-valued coefficients for each term in the constraint polynomials, and C_i are known finite-degree polynomial functions. The constraints C_i and the cost function $C(\mathbf{x})$ can contain terms of any order, e.g., linear, quadratic, cubic, and so on, in the decision variables. All IP problems can be reformulated to satisfy the following conditions using a constant shift to the variables.

- The inequality in the constraint (C_i) is strictly *less* than a certain integer (h_i): $C_i < h_i$.
- The variables $\mathbf{x} \in \mathbf{Z}^+ \cup \{0\}$ in the constraint take *positive* and zero integer values.

This form is suitable for implementing the qudit-based quantum algorithm.

II. QUDIT-BASED QUANTUM ALGORITHM

The qudit-based quantum algorithm for solving the IP problem, as illustrated by the circuit diagram in Fig. 1, consists of two stages divided into seven steps. The steps of the algorithm, as indicated in Fig. 1, are now elaborated.

Stage I. Implementation of the constraints.

Step 1. Initialization. The system is initialized with a d-dimensional qudit register consisting of n qudits representing n integer variables each taking values in $[0, d-1]$. A second register with m qubits is introduced, corresponding to m constraints in the problem, where the state $|1\rangle$ or $|0\rangle$ of each qubit determines if a constraint ($C_i < h_i$) is satisfied or not, respectively. The registers are initialized in the state $|\mathbf{0}\rangle$ for qudits and $|0\rangle$ for qubits, where states in bold denote qudits. Thus, the state of the system with the two registers shown as $|\psi_1\rangle$ in Fig. 1 is,

$$|\psi_1\rangle = |\mathbf{0}\rangle^{\otimes n} |0\rangle^{\otimes m}, \tag{3}$$

where the tensor product $\otimes$ in the state space is implicit.

One advantage of using qudits is that they offer a large Hilbert space d^n where each multi-qudit state naturally encodes the corresponding integer variable assignments. Applying n-Hadamard gates $\hat{H}_d$ with dimension d only on the qudit register results in a uniform superposition of all the possible multi-qudit states, while leaving the qubit register unchanged.

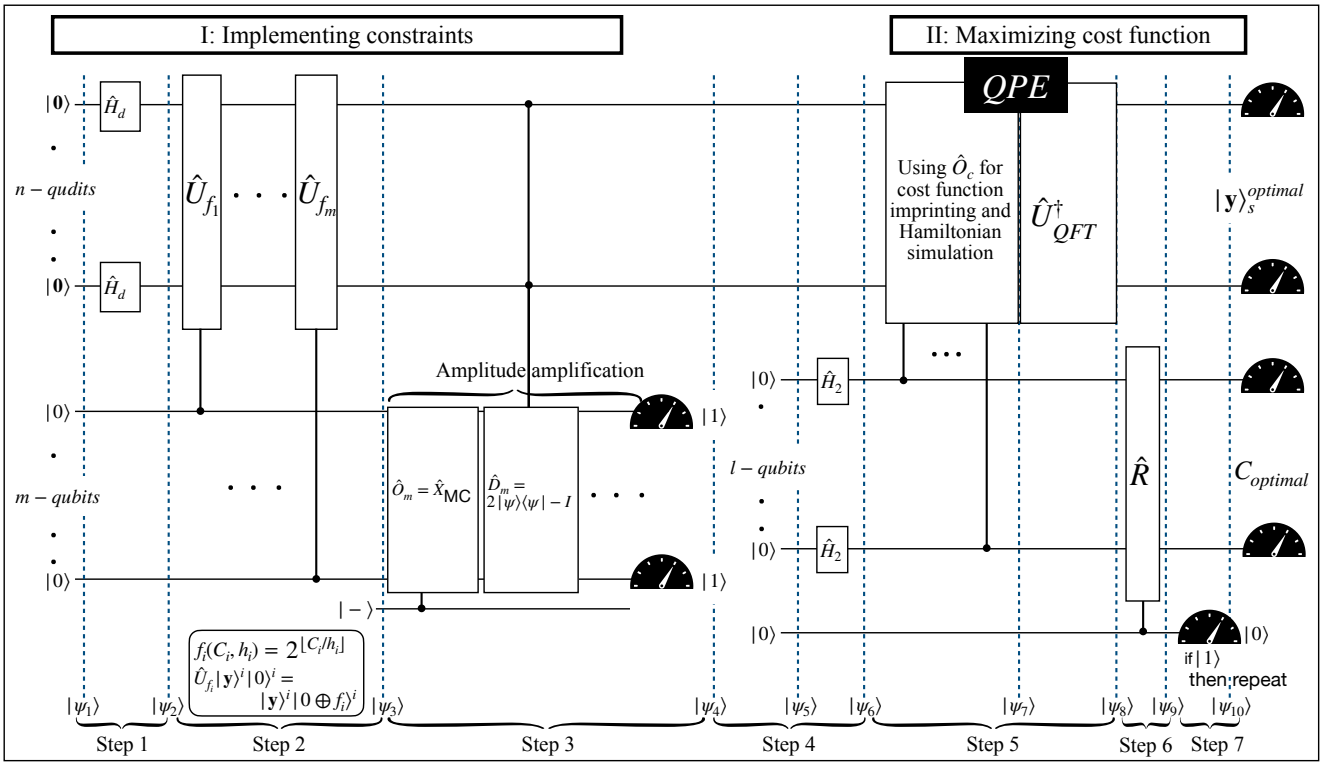


Figure 1: Shows the circuit for the quantum algorithm to solve a general IP problem with n variables and m constraints. It is divided into two stages: implementing constraints and maximizing the cost function. The n-qudit register stores the integer variables, and the m-qubit register indicates the satisfiability of the constraints. The subroutines for the constraints satisfying stage include n-Hadamard gates $\hat{H}_d$ for qudits, m one-sparse entangling unitary operators $\hat{U}_{f_i}$ in sequence, $\sim d^{n/2}$ times application of the Grover operator $\hat{O}_m \cdot \hat{D}_m$ for amplitude amplification, and m-measurements for the qubit register. The next part, namely, maximizing the cost function, begins with a collapsed wave function of the n-qudit superposition state, a qubit register with l qubits, and a single qubit. The subroutines for this part are: l-Hadamard gates $\hat{H}_2$ for qubits, cost function imprinting using a QPE procedure by Hamiltonian simulation of $\hat{O}_c$ (see Step 5 in the main text for the structure of $\hat{O}_c$), a multi-qubit controlled rotation gate $\hat{R}$, and $l+1$-qubit plus n-qudit measurements. The measurements at the end are probabilistic, where the problem is solved with the highest probability compared to other possibilities.

The explicit form of the d-dimensional Hadamard gate is provided in Appendix I. The state of the system is,

$$|\psi_2\rangle = \left(\hat{H}_d^{\otimes n}|\mathbf{0}\rangle^{\otimes n}\right)|0\rangle^{\otimes m} = \left(\frac{1}{d^{n/2}}\sum_{y=0}^{d^n-1}|\mathbf{y}\rangle\right)|0\rangle^{\otimes m}, \tag{4}$$

where $|\mathbf{y}\rangle$ is a shorthand notation for the multi-qudit register defined as,

$$\begin{aligned} |\mathbf{y}\rangle \equiv |\mathbf{x_1}, ..., \mathbf{x_b}, ..., \mathbf{x_n}\rangle = |\mathbf{x_1}\rangle \otimes ... \otimes |\mathbf{x_b}\rangle \otimes ... \otimes |\mathbf{x_n}\rangle \\ \mathbf{x_b} \in \{0, 1, ..., d-1\} \;\; \forall \mathbf{b} \in [1, n], \end{aligned} \tag{5}$$

where $\mathbf{y} \in [0, d^n - 1]$ and each $|\mathbf{y}\rangle$ represents a multi-qudit configuration. As an example, for $n = 2$ and $d = 3$, $\mathbf{y} \in [0, 8]$, where the mapping is such that $|\mathbf{y} = 0\rangle \mapsto |00\rangle$, $|\mathbf{y} = 1\rangle \mapsto |01\rangle$, $\cdots$, $|\mathbf{y} = 8\rangle \mapsto |22\rangle$. The total number of Hadamard gates for the sub-routine $|\psi_1\rangle \mapsto |\psi_2\rangle$ is equal to the number of qudits, i.e., $\sim n$.

Step 2. Implementation of the constraint-satisfying distillation function. One of the main bottlenecks in solving the IP is finding the variable assignments that satisfy all the constraints simultaneously. This is a tedious task as the total space of the variables is exponentially large (d^n), out of which a small subspace needs to be extracted, contributing to the IP problem being NP-hard. The goal is to find a subspace of qudit states satisfying all the constraints of the problem, termed as the *feasible region*. The qubit register plays a crucial role in this procedure as the state $|0\rangle$ / $|1\rangle$ of each qubit determines the corresponding constraint's satisfiability. To extract the constraint information from the qudit register and store it in the qubit register, we define a distillation function f and use it to entangle the two registers. The function f_i corresponding to a constraint $C_i < h_i$ is devised as follows,

$$\begin{aligned} & f_i(C_i, h_i) = 2^{\lfloor \frac{C_i}{h_i} \rfloor} \\ \implies & f_i(C_i, h_i) = \begin{cases} 2^{\lfloor \frac{C_i}{h_i} \rfloor} = \text{even } \mathbf{Z}^+ & : \text{Infeasible region} \\ 2^{\lfloor \frac{C_i}{h_i} \rfloor} = 1 & : \text{Feasible region,} \end{cases} \end{aligned} \tag{6}$$

where $\lfloor C_i/h_i \rfloor$ returns the nearest integer value that is less than C_i/h_i. If the constraint $C_i < h_i$ is satisfied, the value of f_i is

1; otherwise, it's an even integer. The details leading to the construction of the function f_i are provided in Appendix II. An operator definition of the function f_i is required to implement it in a circuit-based algorithm.

The construction of the operator $\hat{U}_{f_i}$ implementing f_i is such that it acts on a restricted subspace of the full qudit Hilbert space. Specifically, it operates on the subset of basis states $|\mathbf{y}\rangle^i$, where only the qudits corresponding to the variables that influence the value of C_i are allowed to change. All other qudits are held fixed. This ensures that $\hat{U}_{f_i}$ modifies only the relevant degrees of freedom needed to evaluate the constraint. The operator $\hat{U}_{f_i}$ divides the set of multi-qudit states $|\mathbf{y}\rangle^i$ into two disjoint subsets; $|\mathbf{y}\rangle^i_s$, consisting of states that satisfy the constraint $C_i < h_i$, and $|\mathbf{y}\rangle^i_k$, consisting of those that do not. $\hat{U}_{f_i}$ entangles all the constraint-satisfying multi-qudit states $|\mathbf{y}\rangle^i_s$ with the qubit state $|1\rangle^i$, and all the other states $|\mathbf{y}\rangle^i_k$ ($\neq |\mathbf{y}\rangle^i_s$) with the qubit state $|0\rangle^i$. Thus, the unitary operator $\hat{U}_{f_i}$ corresponding to the function f_i applied to the set of states $|\mathbf{y}\rangle^i |0\rangle^i$ acts as,

$$\begin{aligned}\hat{U}_{f_i} |\mathbf{y}, 0\rangle^i &= \left|\mathbf{y}, 0 \oplus_2 2^{\lfloor \frac{C_i}{h_i} \rfloor}\right\rangle^i \\ &= \begin{cases} |\mathbf{y}, 0\rangle^i_k & : \text{Infeasible region} \\ |\mathbf{y}, 1\rangle^i_s & : \text{Feasible region.} \end{cases}\end{aligned} \tag{7}$$

where $\oplus_2$ performs Boolean addition.

Since the input and output states corresponding to the unitary $\hat{U}_{f_i}$ are known, the corresponding matrix can be constructed. The unitary $\hat{U}_{f_i}$ turns out to be a 1-sparse matrix; i.e., the matrix consists of only one non-zero element in each row (1-sparse). The 1-sparsity of the matrix is a direct result of Eq. 7, as the operator maps the input state either to itself or permutes it with another state where the qubit has the opposite spin. In general, there are multiple strategies for implementing $\hat{U}_{f_i}$ [48–51]. In our case, we use the permutation matrix to break the unitary into gates, which in the worst case require $O(n^2)$ qudit gates and can be optimized to use $O(n^2 \log d)$ qubit gates [51]. An example of the circuit implementation for $\hat{U}_{f_i}$ is given in Appendix II. Furthermore, for ϵ_u precision of the operator, these 1-sparse unitary matrices can be efficiently implemented using a quantum circuit of polynomial depth $\sim O(n^2 \cdot \log d \cdot \log 1/\epsilon_u)$ [52]. After a sequential application of m such $\hat{U}_{f_i}$ where $i \in [1, m]$ corresponding to m constraints, the qudit states lying in the feasible region of the problem are entangled with the state $|1\rangle^{\otimes m}$ of the qubit register. Thus, the number of gates for m 1-sparse unitary operators is $\sim O(m \cdot n^2 \cdot \log d \cdot \log 1/\epsilon_u)$.

The resulting state after this sequential $\hat{U}_{f_i}$ is,

$$\begin{aligned}|\psi_3\rangle &= \prod_{i=1}^{m} \hat{U}_{f_i} |\psi_2\rangle \\ &= \frac{1}{d^{n/2}} \left(\sum_{(k_\gamma, \gamma)} |\mathbf{y}\rangle_{k_\gamma} |q\rangle_\gamma + \sum_{\mathbf{y}_s \neq \mathbf{y}_{k_\gamma}} |\mathbf{y}\rangle_s |1\rangle^{\otimes m} \right),\end{aligned} \tag{8}$$

where $|q\rangle_\gamma$ is a multi-qubit state such that the corresponding multi-qudit state $|\mathbf{y}\rangle_{k_\gamma}$ satisfies exactly $\gamma \in \{0, \ldots, m-1\}$ of the m constraints. Thus, the first summation runs over pairs (k_γ, γ), where γ = number of satisfied constraints by $\mathbf{y}_{k_\gamma}$.

The state $|q\rangle_\gamma$ is expressed as:

$$|q\rangle_\gamma = \sum_{i=1}^{\binom{m}{\gamma}} c_{\gamma,i} \left|e^\gamma_i\right\rangle, \tag{9}$$

where each $\left|e^\gamma_i\right\rangle$ is an m-qubit computational basis state with Hamming weight γ (i.e., exactly γ qubits in $|1\rangle$, and $m-\gamma$ in $|0\rangle$), while $c_{\gamma,i}$ are the corresponding amplitudes. $\binom{m}{\gamma} = m!/\gamma!(m-\gamma)!$ is the total number of multi qubit basis states. Here, $|q\rangle_0 = |0\cdots 00\rangle$, $|q\rangle_1 = c_{1,1}|0\cdots 01\rangle + c_{1,2}|0\cdots 10\rangle + \cdots + c_{1,m}|1\cdots 00\rangle$, and similarly, $|q\rangle_{m-1} = c_{m-1,1}|1\cdots 10\rangle + c_{m-1,2}|1\cdots 01\rangle + \cdots + c_{m-1,m}|0\cdots 11\rangle$. The detailed steps for applying $\hat{U}_{f_i}$ sequentially to $|\psi_2\rangle$ and give $|\psi_3\rangle$ is provided in Appendix III.

The state $|\mathbf{y}\rangle_s$ satisfies all the m constraints C_i, while $|\mathbf{y}\rangle_{k_\gamma}$ for $\gamma \in [0, m-1]$ are the states that contradict at least one of the constraints. They are defined as,

$$\begin{aligned}|\mathbf{y}\rangle_s &= \sum_{\mathbf{y}^i_s} \left(|\mathbf{y}\rangle^i_s : \; C_j(\mathbf{y}^i_s) < h_j, \forall j \in [1, m] \right) \\ |\mathbf{y}\rangle_{k_\gamma} &= \sum_{\mathbf{y}^i_k} \left(|\mathbf{y}\rangle^i_k : \left| \left\{ j \in [1, m], C_j(\mathbf{y}^i_k) \not< h_j \right\} \right| = \gamma \right)\end{aligned} \tag{10}$$

where $|\cdot|$ is the cardinality of the set, and the expression indicates that the sum (in the bottom equation) is over all the qudit states where exactly γ number of constraints are not satisfied.

Interestingly, the entangled multi-qubit and multi-qudit registers lead to the separation of states with varying degrees of constraint satisfaction. If the feasible region has no solution (undecidable problem), the system can focus on the states entangled with one less constraint to provide the best solution in that case.

Step 3. Reductions to the feasible subspace of IP. The state $|\psi_3\rangle$ (Eq. 8) with a separation of feasible and infeasible states needs to be measured such that the system collapses to the states in the feasible region $|\mathbf{y}\rangle_s$ in one step. Only the measurement of the qubit register needs to be performed such that all the qubits are in the state $|1\rangle$. However, the amplitude of the state $|1\rangle^{\otimes m}$ corresponding to the feasible region $|\mathbf{y}\rangle_s$ may not be maximum. Hence, the goal is to amplify the amplitude of $|1\rangle^{\otimes m}$ by applying Grover's operator for the multi-qubit case. The general procedure for Grover's amplitude amplification requires a combination of the phase flip operator $\hat{O}_m$ and the diffusion operator $\hat{D}_m$, and the detailed description is provided in Appendix IV.

The operator $\hat{O}_m$ applies a phase flip and marks the target state $|1\rangle^{\otimes m}$ with a -1 phase shift while the rest of the states remain unchanged.

$$\begin{aligned}\hat{O}_m |1\rangle^{\otimes m} &= -|1\rangle^{\otimes m} \\ \hat{O}_m |x\rangle &= |x\rangle, \quad \forall |x\rangle \neq |1\rangle^{\otimes m}.\end{aligned} \tag{11}$$

For the implementation of $\hat{O}_m$ in a circuit, an ancilla qubit is introduced and is prepared in $|-\rangle$ state. A multi-controlled

X gate, represented as $\hat{X}_{\mathrm{MC}}$ is applied to the m-qubit register along with $|-\rangle$ which results in the state $(-1)\cdot|1\rangle^{\otimes m}|-\rangle$, marking the state $|1\rangle^{\otimes m}$ with a -1 phase shift. The application of the operator $\hat{O}_m$ to the state $|\psi_3\rangle$ is shown in Appendix IV and the resulting state as follows,

$$\begin{aligned}\hat{O}_m|\psi_3\rangle|-\rangle &= \frac{1}{d^{n/2}}\hat{X}_{MC}\left(\sum_{(k_\gamma,\gamma)}|\mathbf{y}\rangle_{k_\gamma}|q\rangle_\gamma + \sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}|\mathbf{y}\rangle_s|1\rangle^{\otimes m}\right)|-\rangle \\ &= \frac{1}{d^{n/2}}\left(\sum_{(k_\gamma,\gamma)}|\mathbf{y}\rangle_{k_\gamma}|q\rangle_\gamma - \sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}|\mathbf{y}\rangle_s|1\rangle^{\otimes m}\right)|-\rangle\,.\end{aligned} \tag{12}$$

Once the target state $|\mathbf{y}\rangle_s|1\rangle^{\otimes m}$ is marked, the diffusion operator $\hat{D}_m$ is applied, which is used to reflect the state around the average; it is defined in the Appendix IV.

In a circuit, the diffusion operator is implemented using a multi-controlled-Z gate represented as $\hat{Z}_{\mathrm{MC}}$ and m Pauli-X gates denoted by $\hat{X}^{\otimes m}$ in the following way,

$$\begin{aligned}\hat{D}_m &= 2|\psi_3\rangle\langle\psi_3| - I \\ \hat{D}_m &= \hat{U}_f\hat{X}^{\otimes m}\hat{Z}_{\mathrm{MC}}\,\hat{X}^{\otimes m}\hat{U}_f.\end{aligned} \tag{13}$$

where the operator $\hat{U}_f$ is constructed from the tensor product of all $\hat{U}_{f_i}$, and transforms the computational basis states such that the reflection is performed about the state $|\psi_3\rangle$, which is a non-uniform superposition state. This differs from the standard diffusion operator, which is applied to a uniform superposition state and uses Hadamard gates instead of $\hat{U}_f$. The operator $\hat{D}_m$ operates on the complete entangled state $|\psi_3\rangle$. In this transformed basis, the state $|0\rangle^{\otimes m}$ captures the information of the average amplitude of the overall state. The X gate flips each qubit and thereby maps $|0\rangle^{\otimes m}$ state to the target state $|1\rangle^{\otimes m}$. The multi-controlled Z gate $\hat{Z}_{\mathrm{MC}}$ performs the inversion around the mean by applying a phase of -1 to $|0\rangle^{\otimes m}$, which is the target state after the $\hat{X}^{\otimes m}$ operation. The last two operations, $\hat{X}^{\otimes m}$ and $\hat{U}_f$, return the system to the computational basis. This whole sequence reflects all amplitudes around their average and, in turn, amplifies the amplitude of the marked state. Let $P_{\mathbf{y}_s}$ be the probability of measuring the subspace $|\mathbf{y}\rangle_s|1\rangle^{\otimes m}$ in the full state of the system $|\psi_3\rangle$. After one iteration of the Grover operator, the probability increases from $P_{\mathbf{y}_s}$ to $\sin^2(3\cdot\arcsin P_{\mathbf{y}_s}^{1/2})$, which for $P_{\mathbf{y}_s} << 1$ is $\approx 9P_{\mathbf{y}_s}$. For the amplitude amplification to reach probability 1 of measuring the marked entangled state $|\mathbf{y}\rangle_s|1\rangle^{\otimes m}$, one has to take into account both registers.

Thus, the Grover operator $\hat{G}_m = \hat{D}_m\hat{O}_m$ is iteratively applied to the qubit register and keeps amplifying the target state. After p iterations of $\hat{G}_m$, the probability of the target state increases to $\sin^2((2p+1)\cdot P_{\mathbf{y}_s}^{1/2})$. The optimal number of Grover iterations needed to reach the probability 1 of measuring the feasible region is $p_{\mathrm{opt}} \approx (\pi/4)\cdot P_{\mathbf{y}_s}^{-1/2}$. In the worst-case scenario, when there is only one state in the feasible region, $P_{\mathbf{y}_s} = 1/d^n$, leading to the number of iterations $p_{\mathrm{opt}} \approx (\pi/4)\cdot d^{-n/2}$. The number of gates for this operation scales as $O(d^{n/2}\cdot\log\epsilon_G^{-1})$, which is exponential with the number of qudits and logarithmic in the precision ϵ_G [53, 54].

Finally, measuring the system will result in the state $|1\rangle^{\otimes m}$, and the system ends up in the following state,

$$|\psi_4\rangle = F_n\sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}|\mathbf{y}\rangle_s|1\rangle^{\otimes m}\,, \tag{14}$$

where $F_n = N_{\mathbf{y}_s}^{-1/2}$, with $N_{\mathbf{y}_s} \leq d^n$ is the number of multi-qudit states in the feasible region. F_n is the normalizing factor, and the qudit states are collapsed to the feasible region of the problem.

Stage II. Maximizing the cost function.

Step 4. Mid-circuit initialization. After following the steps of the constraint-satisfying stage that takes the multi-qudit states from a large Hilbert space to a relatively smaller subspace $|\mathbf{y}\rangle_s$ forming the feasible region, the subsequent stage is to maximize the cost function $C(\mathbf{y})$ in the feasible region. The constraint register, consisting of m qubits, is obsolete after the measurement. For the optimization procedure, we introduce $l+1$ qubits; all in the state $|0\rangle$, divided into two registers, one with l qubits (further discussion is provided in Step 5) and the other with 1 qubit. The new state of the system is given as,

$$|\psi_5\rangle = F_n\sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}|\mathbf{y}\rangle_s|0\rangle^{\otimes l}|0\rangle\,. \tag{15}$$

To find the maximum cost function, the l-qubit register will store the normalized cost function values, entangled with the qudit register containing the corresponding integer variables, while the single qubit will be used to optimize the cost function.

The Hadamard gates $\hat{H}_2$ as shown in Fig 1, are applied to the l-qubit register to prepare an equal superposition state,

$$\begin{aligned}|\psi_6\rangle &= \hat{H}_2^{\otimes l}|\psi_5\rangle \\ &= F_n\sum_{\mathbf{y}_s}|\mathbf{y}\rangle_s\left(\hat{H}_2^{\otimes l}|0\rangle^{\otimes l}\right)|0\rangle \\ &= \tilde{F}_n\sum_{\mathbf{y}_s}|\mathbf{y}\rangle_s\left(\sum_{k=0}^{2^l-1}|k\rangle\right)|0\rangle\,,\end{aligned} \tag{16}$$

where $\tilde{F}_n = F_n\cdot\sqrt{2^{-l}}$ is the modified normalization factor for the state. l such Hadamard gates are required.

Step 5. Encoding the cost function using quantum phase estimation. The goal is to find the multi-qudit state corresponding to the maximum value of the cost function $C(\mathbf{y}_s)$, which is denoted as $|\mathbf{y}_s^*\rangle$. The cost function value is encoded as the phase of the corresponding multi-qudit state; the phase operator is denoted by $\hat{O}_c$. Any linear or non-linear polynomial-cost function can be implemented by combining Eqs. V-1, V-3 and V-6, given in Appendix V. Here, we choose a non-linear cost function for illustration.

Consider a cost function of the form $C(\mathbf{y}) = a_1x_1x_2 + a_2x_3^2 + a_3x_2^2x_3x_4^2$, where $\mathbf{y} \equiv \{x_1, x_2, x_3, x_4\}$ and $|\mathbf{y}\rangle = |x_1, x_2, x_3, x_4\rangle$. The phase operator $\hat{O}_c$ encoding the cost function using

$\hat{q}\left|\mathbf{x_j}\right\rangle = x_j\left|\mathbf{x_j}\right\rangle$ from Eq. V-1 is given, as follows,

$$\begin{aligned}
&\hat{O}_c\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle \\
&\quad= e^{i2\pi a_3(\hat{I}\otimes\hat{q}^2\otimes\hat{q}\otimes\hat{q}^2)/C_{ub}}e^{i2\pi a_2(\hat{I}\otimes\hat{I}\otimes\hat{q}^2\otimes\hat{I})/C_{ub}}e^{i2\pi a_1(\hat{q}\otimes\hat{q}\otimes\hat{I}\otimes\hat{I})/C_{ub}} \\
&\qquad\left(\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle\right) \\
&\quad= e^{i2\pi a_3 x_2^2 x_3 x_4^2/C_{ub}}e^{i2\pi a_2 x_3^2/C_{ub}}e^{i2\pi a_1 x_1 x_2/C_{ub}} \\
&\qquad\left(\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle\right) \\
&\quad= e^{i2\pi(a_1x_1x_2+a_2x_3^2+a_3x_2^2x_3x_4^2)/C_{ub}}\left(\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle\right) \\
&\quad= e^{i2\pi C(\mathbf{y})/C_{ub}}\left(\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle\right).
\end{aligned} \tag{17}$$

where the operator is applied only to the states in the qudit register and the quantity $C(\mathbf{y}_s)/C_{ub} < 1$. C_{ub} is the upper bound for the cost function required to make sure the phase lies in $[0, 2\pi]$, which can be calculated in polynomial time using classical algorithms.

For the first term in the cost function $a_1x_1x_2$, the phase operator is defined using Eq. V-1 that is $\hat{q}\left|\mathbf{x_j}\right\rangle = x_j\left|\mathbf{x_j}\right\rangle$ and the properties given by Eqs. V-3 and V-6. The operator is thus,

$$\begin{aligned}
&e^{i2\pi a_1(\hat{q}\otimes\hat{q}\otimes\hat{I}\otimes\hat{I})/C_{ub}}\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle \\
&\quad= e^{i2\pi a_1x_1x_2/C_{ub}}\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle.
\end{aligned} \tag{18}$$

The second term of the cost function $a_2x_3^2$ requires an additional application of $\hat{q}$, leading to $\hat{q}\hat{q}\left|\mathbf{x_j}\right\rangle = x_j^2\left|\mathbf{x_j}\right\rangle$, the corresponding operation is as follows,

$$\begin{aligned}
&e^{i2\pi a_2(\hat{I}\otimes\hat{I}\otimes\hat{q}^2\otimes\hat{I})/C_{ub}}\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle \\
&\quad= e^{i2\pi a_2x_3^2/C_{ub}}\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle.
\end{aligned} \tag{19}$$

For the implementation of the last term $a_3x_2^2x_3x_4^2$, all of the above properties are utilized to give,

$$\begin{aligned}
&e^{i2\pi a_3(\hat{I}\otimes\hat{q}^2\otimes\hat{q}\otimes\hat{q}^2)/C_{ub}}\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle \\
&\quad= e^{i2\pi a_3x_2^2x_3x_4^2/C_{ub}}\left|\mathbf{x_1}\right\rangle\otimes\left|\mathbf{x_2}\right\rangle\otimes\left|\mathbf{x_3}\right\rangle\otimes\left|\mathbf{x_4}\right\rangle.
\end{aligned} \tag{20}$$

An example for defining the operator $\hat{O}_c$ corresponding to a linear cost function is given in Appendix V along with its application to the multi-qudit state of the system.

The phase operator is used to perform a quantum phase estimation (QPE) procedure using Hamiltonian simulation of $\hat{O}_c$, which operates on the superposition state $|\psi_6\rangle$. This stores the phase (the normalized cost function) corresponding to each multi-qudit state in the l-qubit register (QPE register). Then, a controlled rotation of the single qubit converts the eigenvalues of the l-qubit register to the amplitude of the state and finds the optimal solution, which will be further detailed in Step 6. Our approach is inspired by the HHL algorithm [55]. However, it differs from the HHL in terms of the definition of the phase operator $\hat{O}_c$, leading to the procedure of encoding non-linear terms in the phase. The QPE operator is given by,

$$\sum_{k=0}^{2^l-1}|k\rangle\langle k|\otimes\hat{O}_c^k \text{ with } \hat{O}_c^k\left|\mathbf{y}\right\rangle_s = (e^{i2\pi\tilde{\phi}(\mathbf{y}_s)})^k\left|\mathbf{y}\right\rangle_s.$$

Applying this operator to Eq. 16, results in,

$$\begin{aligned}
|\psi_7\rangle &= \left(\sum_{k=0}^{2^l-1}|k\rangle\langle k|\otimes\hat{O}_c^k\right)|\psi_6\rangle \\
&= \tilde{F}_n\sum_{\mathbf{y}_s}\sum_{k=0}^{2^l-1}e^{i2\pi\tilde{\phi}(\mathbf{y}_s)k}\left|k\right\rangle\left|\mathbf{y}\right\rangle_s|0\rangle,
\end{aligned} \tag{21}$$

where $\tilde{\phi}(\mathbf{y}_s) = \phi(\mathbf{y}_s) + 1/C_{ub}$ and $\phi(\mathbf{y}_s) = C(\mathbf{y}_s)/C_{ub}$. $C_{ub} = C_{cont} + 1.5$ where C_{cont} is calculated in polynomial time using classical algorithms by solving the relaxed problem (integer to continuous variables). The maximum value of $C(\mathbf{y}_s)$ is C_{cont}, leading to $\phi(\mathbf{y}_s) \in [0, 1)$ and $\tilde{\phi}(\mathbf{y}_s) \in (0, 1)$. By construction, the values of $\tilde{\phi}(\mathbf{y}_s)$ are strictly less than 1 to make sure that no information is lost during the QPE routine. This is because the eigenvalues of any phase unitary operator lie on a unit circle and are inherently periodic; that is, for integer p, $e^{i2\pi\theta} = e^{i2\pi(\theta+p)}$. Hence, in QPE, the integer part of the phase is not stored in the qubit register. The lower bound on $\tilde{\phi}(\mathbf{y}_s)$ is made strictly greater than zero by adding $1/C_{ub}$ to $\phi(\mathbf{y}_s)$; the reason will be reflected later during the controlled single qubit rotation step. The complexity for the QPE procedure is $O(n\cdot\epsilon_{QPE}^{-1})$, where ϵ_{QPE} is the precision for the QPE operation [53, 56].

The application of the operator $\hat{O}_c$ to $|\psi_6\rangle$ reads,

$$\begin{aligned}
\left|\tilde{\psi}_6\right\rangle &= \hat{O}_c|\psi_6\rangle \\
&= \tilde{F}_n\left(\sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}e^{i2\pi\tilde{\phi}(\mathbf{y}_s)}\left|\mathbf{y}\right\rangle_s\right)\left(\sum_{k=0}^{2^l-1}|k\rangle\right)|0\rangle,
\end{aligned} \tag{22}$$

which is used in Eq. 21.

Lastly, for the QPE routine, the inverse QFT operation represented as $\hat{U}_{QFT}^\dagger$, is performed to extract the value of the phase into the l-qubit register, given as

$$\begin{aligned}
\hat{U}_{QFT}^\dagger|k\rangle &= \frac{1}{\sqrt{2^l}}\sum_{j=0}^{2^l-1}e^{-2\pi i\frac{jk}{2^l}}|j\rangle \\
\implies |\psi_8\rangle &= \hat{U}_{QFT}^\dagger|\psi_7\rangle \\
&= F_n\sum_{\mathbf{y}_s}\left|\tilde{\phi}(\mathbf{y}_s)\right\rangle^{\otimes l}\left|\mathbf{y}\right\rangle_s|0\rangle,
\end{aligned} \tag{23}$$

which requires $O(l^2)$ gate operations [57]. $\left|\tilde{\phi}(\mathbf{y}_s)\right\rangle^{\otimes l}$ is the l-qubit superposition state which sharply peaks at the binary representation of the corresponding value of the phase $\tilde{\phi}(\mathbf{y}_s)$. The operator $\hat{U}_{QFT}^\dagger$ is implemented using Hadamards and controlled rotation gates [58].

Step 6. Amplitude encoding of the cost function. The qubit register in Eq. 23 encodes the cost function values. To find

the maximum one, the eigenvalues are encoded in the amplitude of the full state. This is done by a controlled rotation $\hat{R}$ on the ancilla qubit based on the eigenvalue of the l-qubit register. The inverse of the eigenvalues $\tilde{\phi}(\mathbf{y}_s)$ ($\neq 0$) is encoded in the amplitude of the qubit state as $C/\tilde{\phi}(\mathbf{y}_s)$ for the $|1\rangle$ and $\sqrt{1 - C^2/\tilde{\phi}(\mathbf{y}_s)^2}$ for $|0\rangle$, where C is a normalization constant for the rotation, which is chosen to be $1/C_{ub}$. The resulting state of the system is given as,

$$\begin{aligned} |\psi_9\rangle &= F_n \sum_{\mathbf{y}_s} |\mathbf{y}\rangle_s \left|\tilde{\phi}(\mathbf{y}_s)\right\rangle^{\otimes l} \hat{R}(\tilde{\phi}(\mathbf{y}_s)) |0\rangle \\ &= F_n \sum_{\mathbf{y}_s} |\mathbf{y}\rangle_s \left|\tilde{\phi}(\mathbf{y}_s)\right\rangle^{\otimes l} \left(\sqrt{1 - \frac{1}{(1 + C(\mathbf{y}_s))^2}} |0\rangle \right. \\ &\qquad \left. + \frac{1}{(1 + C(\mathbf{y}_s))} |1\rangle \right). \end{aligned} \tag{24}$$

The complexity for this rotation is $O(\log \epsilon_R^{-1})$ for ϵ_R precision. Here, the amplitude corresponding to the qubit state $|1\rangle$ is inversely proportional to the value of the cost function $C(\mathbf{y}_s)$; hence, the amplitude of the state $|0\rangle$ increases with $C(\mathbf{y}_s)$. The integer variables that maximize the cost function are encoded in the qudit states, while the cost function value is encoded in the l-qubit state entangled with the $|0\rangle$ of the ancilla qubit.

Step 7. Final measurement for the optimal solution. The amplitude of the states in Eq. 24 is dependent on the value of the cost function of the problem. The measurement of the ancilla qubit with the desired state being $|0\rangle$ (pre-factor) provides the optimal solution with relatively high probability. If the measurement leads to $|1\rangle$, then the procedure has to be repeated until the state $|0\rangle$ is measured. The state after the measurement is,

$$|\psi_{10}\rangle = \frac{F_n}{\sqrt{p_0}} \sum_{\mathbf{y}_s} \sqrt{1 - \frac{1}{(1 + C(\mathbf{y}_s))^2}} \cdot |\mathbf{y}\rangle_s \left|\tilde{\phi}(\mathbf{y}_s)\right\rangle^{\otimes l} \tag{25}$$

where $p_0 = \sum_{\mathbf{y}_s} \left(1 - 1/(1 + C(\mathbf{y}_s))^2\right)$. The beauty of our construction is such that a finite set of measurements of the l-qubit register will correspond to the optimal cost function value, with relatively high probability due to the phase-amplitude encoding. This can be observed from the equation above. Before Stage II, all the states in the collapsed qudit register that satisfy the constraints are uniformly distributed, each with amplitude $N_{\mathbf{y}_s}^{-1/2}$, where $N_{\mathbf{y}_s}$ is the number of such states. However, the amplitude of the state corresponding to the minimum value of the cost function $C(\mathbf{y}_s) = 0$ becomes zero. This amplitude is distributed with a bias on the maximum cost function value among the remaining states, which maximizes the probability of the optimal solution. The qudit register collapses to the state encoding the optimal variables after this measurement. This is discussed further in the next section.

III. NUMERICAL DEMONSTRATION OF THE QUDIT-BASED QUANTUM ALGORITHM

The quantum algorithm presented in this work can tackle any linear and non-linear IP problem; however, non-linear problems are much harder to solve classically [60]. This is because often non-linear terms in constraints make the optimization non-convex [61]. Hence, for a demonstration that our quantum algorithm can tackle classically hard problems, we chose a sample non-linear IP given as,

$$\begin{aligned} &C(\mathbf{x}) = 2x_1 + x_2 + x_3 + 3x_4 + \frac{3}{2}x_5 \\ &\text{To be maximized under:} \\ &C_1 : x_1 + x_2^2 x_3 + x_3 < 1 \\ &C_2 : 3x_3^2 x_4 + x_2 < 2 \\ &C_3 : x_1 x_5 + x_4 < 1 \\ &C_4 : 2x_1 + 2x_1^2 x_3 + x_4^3 < 2 \\ &x_i \in \{0, 1, 2\}. \end{aligned} \tag{26}$$

and solve it using our algorithm. The problem contains quadratic and cubic terms, making it difficult for general-purpose gradient-based and interior-point methods to solve [62], precisely due to the non-convexity of the problem. Often, the brute-force method is preferred for solving such problems. The non-convexity of the problem is analyzed in Appendix VI. The algorithm is implemented using Python [63], where each operator shown in Fig. 3 is well-defined mathematically, leading to the execution of the corresponding circuit architecture for solving the above IP problem.

Fig. 2 shows the result of the implementation of the quantum algorithm for solving this non-linear non-convex optimization problem. The probability distribution of the constraint register in panel (a) and the data register in panel (b) is shown during the algorithmic run. The algorithm initialises with 5-qudits corresponding to 5 variables, each taking 3-values and 4-qubits for the constraint register. The total number of qudit states is $3^5 = 243$. In panel (a), the probability of the basis of many-qubit states (numbered from 0 to $2^4 - 1$) is shown, where each basis has two bars corresponding to the probability after applying entangling unitary operators U_{f_i} (brown bars) and after the amplitude amplification step (green bars) respectively. The unitary operators (as described in Step 2 in Sec. II) give bias to some of the basis states, $|0\rangle$ in this case, indicating that many of the qudit states in the superposition are entangled with $|0\rangle$, a consequence of a large infeasible region. To amplify the qubit state $|1111\rangle$ (represented as $|15\rangle$ in panel (a) of Fig. 2) entangled with the feasible region (all constraints satisfied), a Grover operator is applied (Step 3 in Sec. II), maximizing the probability of the target state, as shown by the green bar.

After measuring the qubit register, the corresponding qudit register collapses to the states satisfying all the constraints simultaneously, which amounts to a total of 6 out of $3^5 (= 243)$ states as shown in panel (b). The brown bars in panel (b) show an equal superposition of the states in the feasible region, with the basis being numbered. The goal now is to find

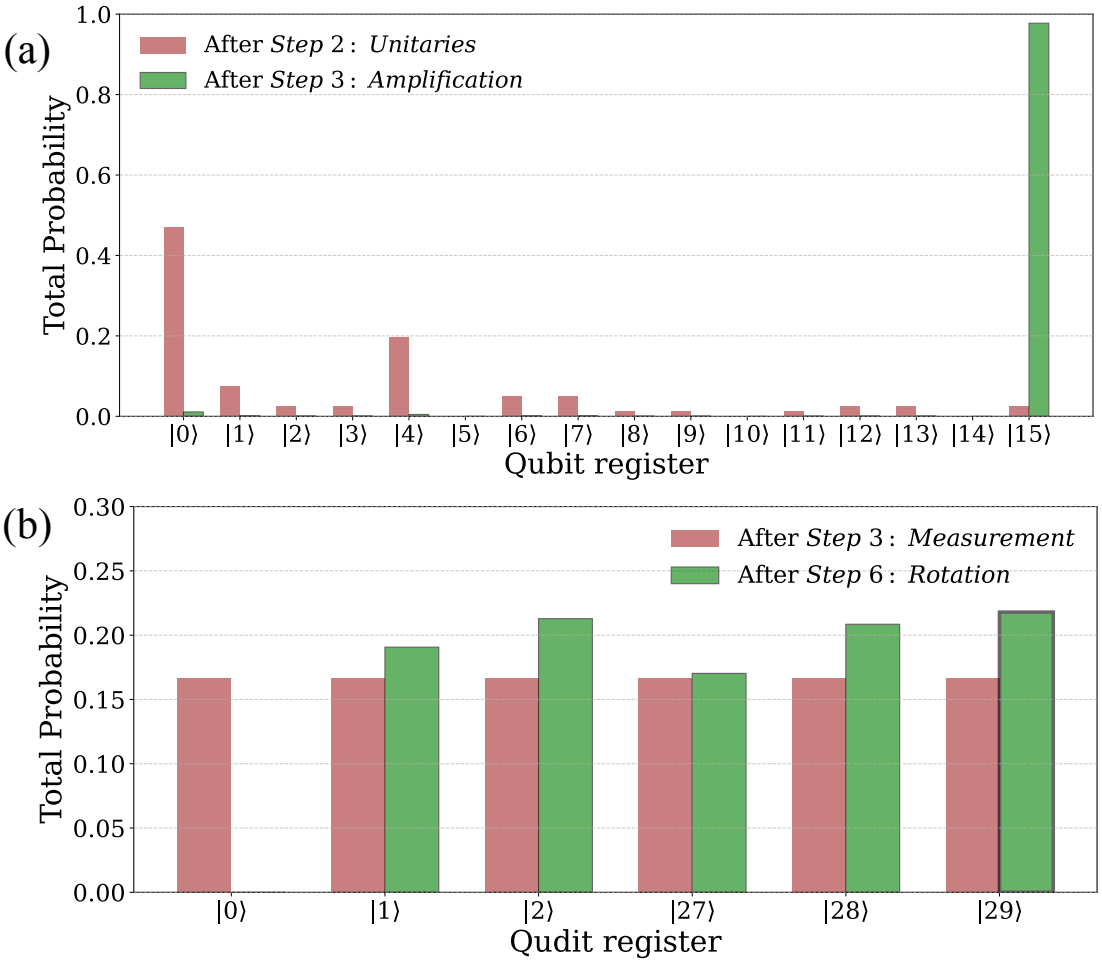


Figure 2: Quantum solution of a non-linear integer program with 5 variables and 4 non-convex constraints. In both panels, the states on the x-axis are numbered basis states, for (a) it ranges from 0 to $2^4 - 1$, and for (b) only the states in the feasible region are shown. (a) Shows qubit register (constraint) probabilities before (brown) and after (green) amplitude amplification, highlighting the amplification of the feasible state $|15\rangle$. (b) Shows qudit register probabilities before (brown) and after (green) cost function optimization via QPE, with the optimal solution $|\mathbf{29}\rangle$ (cost = 4) amplified. The quantum algorithm finds the solution using ≤ 19 repetitions.

the one state that maximizes the cost function. As described in Sec. II, a QPE procedure with $l = 4$ qubits is performed, and another ancilla qubit is rotated based on the QPE register. This results in the final redistributed probabilities in the feasible region, as shown by green bars in panel (b). There are three key messages here: (i) the probability of the optimal cost function is always higher than the rest, indicated by the highlighted green bar with black cover, (ii) the number of repetitions r calculated using Eq. 34 for $p = 0.22$ to guarantee a success with $P_{target} = 0.99$ is at most 19, (iii) The probability of the qudit state $|\mathbf{0}\rangle$ after the cost function optimization procedure goes to zero, and this excess probability is distributed among the other states with a bias based on the cost function value. In this case, the state $|\mathbf{29}\rangle \equiv |01002\rangle$ is the optimal solution with $C(x) = 4$.

For comparison, a branch and bound method is used to solve the same problem, which provides the optimal solution; however, it requires 358 nodes (computations). The brute-force method requires $3^5 (= 243)$ computations, which is less than that of the branch and bound algorithm. We also solved the problem using a brute force approach and Google-OR-tools CP-SAT solver [69]; the brute force required one-tenth of the time to reach the solution compared to the CP-SAT solver. This indicates that as the non-convexity in the problem increases, brute force becomes better than the available general-purpose solvers. Hence, for such integer programming problems, it suffices to compare the time complexity with the brute-force method. However, the mentioned classical methods (other than the brute-force) do work well for convex problems. We compare next the time complexity of various special-purpose classical algorithms with our algorithm, where a discussion of the limitations of classical algorithms is also presented.

In Fig. 3, the time complexity scaling of multiple classical algorithms, along with the brute force approach, is shown and compared with our quantum algorithm. The classical ex-

Our Algorithm works for **any IP** with time scaling $\mathbf{T_Q \sim mn^2 \log d + 6 \cdot d^{n/2} + n\epsilon_{QPE}^{-1}}$ to give **exact** solutions				
Class	**Method**	**Time Scaling** T_{Cl}	**Type**	**Speed-up criterion of our algorithm** $T_{Cl}/T_Q > 1$
Linear IP	Lenstra (1983)	$2^{n^3} \cdot m^3 \log^2 V$	Exact	$\sim 2^{n^3}/d^{n/2}$: Speed-up for $n \geq \sqrt{\log \sqrt{d}}$
Linear IP	Kannan (1987)	$n^{2n} e^{O(n)} \cdot m^2 \log^2 V$	Exact	$\sim n^{2n} \cdot e^{O(n)}/d^{n/2}$: Speed-up for $n > (d/e^{O(1)})^{1/4}$
Linear IP	Frank-Tardos (1986)	$n^{2.5n} \cdot 2^{2n} \cdot m^2 \log^2 V$	Exact	$\sim n^{2.5n} \cdot 2^{2n}/d^{n/2}$: Speed-up for $n > (d/4)^{1/5}$
Linear IP	Dadush (2011)	$n^n \cdot 2^n \cdot m^2 \log^2 V$	Exact	$\sim n^n \cdot 2^n/d^{n/2}$: Speed-up for $n > \sqrt{d/4}$
Linear IP	Reis-Rothvoß (2023)	$(\log n)^{3n} \cdot m^4 \log^3 V$	Exact	$\sim \log n^{3n}/d^{n/2}$: Speed-up for $n > e^{d^{1/6}}$
Convex IP	Convex IP (fixed dim)	$(n \log V)^{2n} \cdot m^3$	Exact	$\sim n^{2n}/d^{n/2}$: Speed-up for $n > d^{1/4}$
Any IP	Brute force	$d^n \cdot m \cdot n^k \cdot \log^c L$	Exact	$d^{n/2}$: Speed-up for all n
Linear IP	LP + Rounding	$m^{3.5} n^2 \log^2 V$	**Approximate**	Speed up only for small d and small n, $d < n^{4/n}$
General non-linear IP[a]	Heuristics	Undecidable/EXP-hard	–	Always (no efficient classical algorithm)[b]

[a] There are algorithms for convex non-linear IP with scaling $\sim n^{2n}$ and no algorithm with provable time complexity for a general non-convex NLIP [60].
[b] Our algorithm incurs a speed-up when the problem instance is decidable, while for undecidable instances, a set of finite measurements of the qubits in the constraint register can effectively provide the information. Refer to the Results section for further explanation.

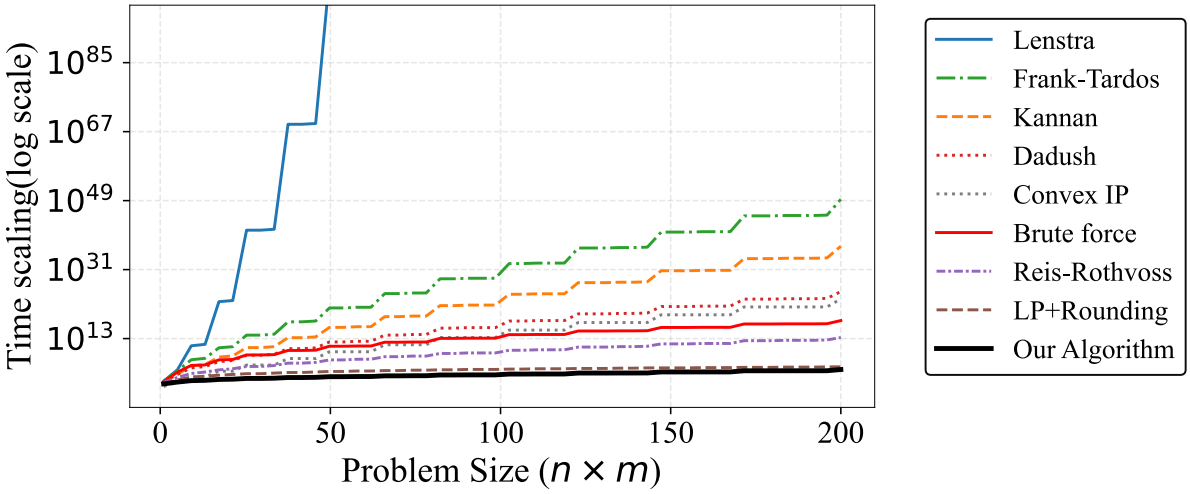


Figure 3: Comparative analysis of integer programming algorithms. Top: Time complexity expressions for each method, along with the class of problems that can be solved. The table also shows the criterion when our algorithm has a speed-up over the classical cases. For most of the algorithms, as n increases, the quantum algorithm becomes faster, except for the approximate LP+Rounding, which is used to solve linear problems. The approximation ratio for LP+rounding algorithms is bounded by the integrality gap (ratio of the optimal solution of the relaxed-continuous variable problem and the IP problem), which is 2 for the vertex cover problem, $\ln n$ for the set cover problem, and 3/4 for the Max-SAT problem [59]. All logarithms are base-2. V is the maximum coefficient magnitude, k is the degree of the polynomials, c is the bit-precision, and ϵ_{QPE} is the quantum phase estimation precision. Bottom: Time complexity scaling with problem size ($n \times m$) with $d = 3$, $V = 10$, $c = 2$, $k = 3$ and $\epsilon_{QPE} = 0.1$.

act algorithms presented in the top table show a provable time scaling with various parameters of the IP problem; however, the methods are limited to linear IPs [70–76]. This is due to these algorithms relying on the optimization landscape being a convex polytope, which allows them to use geometric methods, such as using ellipsoids to find the solution. On the other hand, approximate algorithms for solving IP show polynomial time scaling with problem size, but with multiple assumptions, such as in FPTAS [77], only a single linear constraint with a quadratic cost function is allowed, or in LP+rounding, it fails for non-linear constraints and suffers from rounding off errors. There are multiple commercially available solvers for IP, such as Gurobi, CPLEX, BARON, Couenne, and IOPT [78–80], with underlying algorithms being either one or a combination of branch and bound, cutting planes, and heuristics, coupled with the relaxation of the integer constraint. To solve IP using these algorithms, a relaxation step is used to convert the integer variables to continuous, and then the relaxed problem is solved with modifications in each iteration. Most of these commercially available classical solvers also have a few restrictions on the class of problems they can solve efficiently, such as the constraints must be linear and the inequalities must be non-strict. The non-strict inequalities (such as $C_i \leq h_i$ instead of $C_i < h_i$) are to make sure that there are no open sets in the optimization polytope. The solution of the relaxed problems resides on the boundaries of this polytope, and open sets remove the best guess for the relaxed problems. The table in Fig. 3 also indicates the threshold where our algorithm shows a speed-up over the classical algorithms listed. The brute force algorithm starts performing better than the other classical algorithms for higher n as well; however, it always scales worse than the quantum algorithm

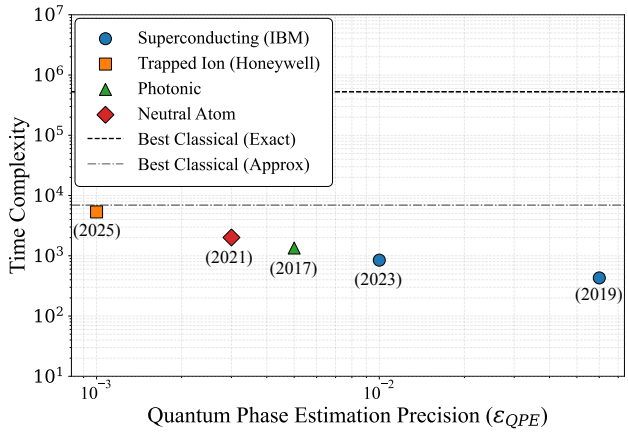


Figure 4: Comparison of time complexity versus quantum phase estimation precision ϵ_{QPE} for our quantum algorithm across various experimental platforms: Superconducting (2019,2023) [64, 65], Trapped Ion (2025) [66], Photonic (2017) [67], and Neutral Atom (theory with experimental parameters, 2021) [68]. Each data point corresponds to reported ϵ_{QPE} values and demonstration years. The time complexity, computed as $mn^2 \log_2 d + d^{n/2} + \frac{n}{\epsilon_{\text{QPE}}}$ with $m = n = d = 5$, is plotted on a log-log scale. Horizontal dashed and dash-dot lines represent the complexities of the best-known classical exact algorithm (Reis-Rothvoss) and the best approximate classical algorithm (LP + rounding), respectively.

(d^n vs $(\sqrt{d})^n$), and its time complexity increases further with the degree of the polynomial k (non-linearity of the IP), leading to a larger advantage in our approach. The last row of the table indicates that, in practice, heuristics are often used to solve a given non-convex IP problem, but they fail to guarantee the optimal solution or provide an analytical time scaling. The time complexity for the quantum algorithm provided here is for the worst-case scenario; hence, in practice, it will always perform better.

In the bottom panel in Fig. 3, we compare the time complexity of the various algorithms listed in the top table with our algorithm as a function of problem size $n \times m$, as both n and m contribute to the complexity of the problem. For the plot, the dimension of the variables is $d = 3$, the largest coefficient in the problem V is chosen to be 10, and ϵ_{QPE} is 0.1, which is typical in digital quantum circuits. V is the maximum coefficient that appears either in the constraints or the cost function, for example, in Eq 26, $V = 3$; the scaling of many algorithms depends on this parameter due to all the numbers being stored in bits. For computational purposes, a threshold of 10^{100} for the y-axis is fixed, as it is expensive to show further. At $n \times m = 50$, the Lenstra algorithm reaches this threshold. The scaling of our quantum algorithm with problem size is by far the best compared to the other exact algorithms, and it beats the approximate algorithms in terms of the class of problems our algorithm can tackle. This is also the best-known time scaling of an algorithm for non-linear polynomial-IP problems, which in the classical case is EXP-hard/Undecidable [60, 81]. The undecidable problems are those that do not guarantee whether a solution exists or not, and the algorithms may get stuck forever in a loop if such a problem is encountered. One of the features of our algorithm is that there is an intermediate measurement step after the constraint-satisfying stage, which helps to mitigate such infinite loops practically. If in that step, the probability of measuring $|1\rangle^{\otimes m}$ is zero, the amplitude amplification will not result in the state $|1\rangle^{\otimes m}$, leading to the conclusion that the feasible region for the given constraints does not exist. In that case, the computation can be stopped after a few measurements, and the problem instance can be declared undecidable. For practical purposes, this can save computational resources and identify the difficulty of solving the given problem instance. The output of the measurement of the constraint register also contains information about the states that satisfy a range of constraints, which can help in identifying the maximum number of constraints required to make the problem decidable. For example, if probability of $|1\rangle^{\otimes m}$ is zero and the next non-zero probability multi-qubit state has one of the qubits is in $|0\rangle$ while the rest are in $|1\rangle$, then the problem may become decidable if one of the constraints is removed. Counting the number of zeros in the subsequent multi-qubit states provides the crucial information for making the problem decidable. Next, we analyse our algorithm further to understand the limits for achieving a quantum advantage.

Fig. 4 analyses the time complexity of our quantum algorithm with the precision ϵ_{QPE} reached by various quantum computing platforms. The plot considers superconducting (experiment), trapped ion (experiment), photonic (experiment), and neutral atoms (theory with experimental considerations) quantum computing platforms [64–68], where ϵ_{QPE} is characterized by the errors in the gates and the number of qubits in the register used for the experiment. So far, there have been a few experiments for QPE, and for the platforms where the precision is not explicitly provided, it is estimated by the number of qubits used in the procedure. For l-qubits in QPE, the smallest phase that can be determined is given by $1/2^l$, which defines the precision for the procedure. In general, as the number of error-corrected qubits increases, the precision gets better.

For the plot, the following parameters were chosen: $n = m = d = 5$, and for classical algorithms, $V = 10$. The two horizontal lines show the time complexity of the best exact classical algorithm and the best approximate classical algorithm, indicating that if a problem with such parameters is considered, the quantum algorithm is still better. It is also crucial to the quantum algorithm that an exponentially precise phase estimation is not a strict requirement, as the separation of the relative phases between the multi-qudit states is enough to maximize the optimal cost function. The only condition to be satisfied is given by Eq. 32, which ensures that the quantum algorithm provides a valid solution. The smaller the RHS of Eq. 32, the better it is for the quantum algorithm. However, a smaller ϵ_{QPE} leads to higher time complexity; for all practical purposes, $\epsilon_{QPE} \sim 10^{-3}$ [82] is a good compromise for the given parameters. If the problem size increases, the classical algorithms scale worse as shown in Fig. 3, allowing for a

smaller ϵ_{QPE} to retain the quantum advantage. If a non-linear problem such as the one given by Eq. 26 is solved, then the classical algorithms will most likely fail. However, heuristics that perform global optimization for nonlinear and commercial solvers can still be comparable to the quantum algorithm, but for a limited class of problems.

IV. COMPLEXITY ANALYSIS AND THE SUCCESS PROBABILITY OF THE ALGORITHM

Complexity analysis. Table I presents a detailed breakdown of the quantum resources and time complexity associated with each stage of our quantum algorithm for solving the IP problem [52–54, 56, 58]. Each step corresponds to a subroutine(s), where resources are categorized by gate type (single-/multi-qubit and single-/multi-qudit). And the time complexity is analysed in terms of the number of integer variables n, constraints m, qudit dimension d, QPE register size l, and algorithmic precision parameters ϵ_u, ϵ_G, ϵ_{QPE}, and ϵ_R.

The algorithm begins with state initialization in Step 1, where n single-qudit gates are in a uniform superposition of the input variables, which scales linearly with n, with a time complexity of $O(n)$. Step 2 encodes constraints through sparse unitary operators, requiring $O(m)$ multi-qudit gates. There is a logarithmic dependency on the dimension and precision, leading to the overall time complexity being $O(m{\cdot}n^2{\cdot}\log d{\cdot}\log \epsilon_u^{-1})$. Step 3 is a computationally expensive subroutine: amplitude amplification, requiring exponential resources. Specifically, it involves $\sim 6 \cdot d^{n/2}$ single-qudit and $\sim d^{n/2}$ multi-qudit gates, resulting in a time complexity of $O(d^{n/2} \cdot \log \epsilon_G^{-1})$. This step represents the algorithm's bottleneck in terms of gate counts, time complexity scaling, and the number of measurements required.

In Step 4, a series of l single-qubit gates are applied, scaling linearly as $O(l)$, for QPE preparation. In Step 5, Quantum Phase Estimation (QPE) is applied, using multi-qudit and multi-qubit gates. The associated time complexity is $O(n \cdot \epsilon_{QPE}^{-1})$, showing the sensitivity of the process to precision. This step also consists of the inverse QFT procedure, with $O(l^2)$ time scaling. Step 6 implements a rotation, requiring only a single multi-qubit gate and an associated logarithmic cost $O(\log \epsilon_R^{-1})$. Steps 7 incur no resource cost.

The total time complexity across all steps is

$$O\Big(n + m \cdot n^2 \cdot \log d \cdot \log \epsilon_u^{-1} + d^{n/2} \cdot \log \epsilon_G^{-1} + l + l^2 + n \cdot \epsilon_{QPE}^{-1} + \log \epsilon_R^{-1}\Big). \tag{27}$$

In leading-order terms, neglecting lower-order constants and considering a small QPE register (low l), the time simplifies to:

$$O\left(m \cdot n^2 \cdot \log d + d^{n/2} + n \cdot \epsilon_{QPE}^{-1}\right).$$

This expression highlights the key contributions to the overall complexity: the encoding of the constraints, the exponential cost of amplitude amplification, and the precision overhead of the QPE procedure. Since the algorithm depends on the size of the feasible region, let $N_{\mathbf{y}_s} \in [1, d^n]$ denote the size of the feasible region, leading to the time complexity of:

$$O\left(m \cdot n^2 \cdot \log d + \frac{d^{n/2}}{\sqrt{N_{\mathbf{y}_s}}} + n \cdot \epsilon_{QPE}^{-1}\right).$$

The upper bound on $N_{\mathbf{y}_s}$ in our algorithm can be estimated from the unitary operators describing the distillation function, by counting the number of off-diagonal elements in each $\hat{U}_{f_i}$, giving,

$$N_{\mathbf{y}_s} < \frac{1}{2} \min_i \#\left\{(j,k) \mid j \neq k,\ [\hat{U}_{f_i}]_{jk} \neq 0\right\},$$

where the symbol # denotes 'number of elements'. The $\hat{U}_{f_i}$ containing the minimum number of off-diagonal elements corresponds to twice the maximum number of qudit-states that can satisfy all the constraints.

Success probability. The process of resolving the optimal cost function is determined by the accuracy and the precision of the QPE procedure (thus the choice of quantum platform) that modifies the final probabilities before measurements [53, 83, 84]. Here, we provide bounds on the success probability depending on the accuracy of QPE and a relationship between the precision and the IP problem to gain more insights.

Let $\mathcal{S} \subset \mathbb{Z}^n$ denote the feasible region with $N_{\mathbf{y}_s} = |\mathcal{S}|$, and let $C(\mathbf{y}_s) : \mathcal{S} \to \mathbb{R}_{\geq 0}$ be the cost function. $C_E(\mathbf{y}_s)$ is the QPE output of the cost function value $C(\mathbf{y}_s)$. Ideally, the quantum state at the end of the protocol should be given by Eq. 24. However, due to errors in QPE, the output state will contain $\tilde{\phi}_E(\mathbf{y}_s) = (C_E(\mathbf{y}_s) + 1)/C_{ub}$ as the QPE estimate of $\tilde{\phi}(\mathbf{y}_s) = (C(\mathbf{y}_s) + 1)/C_{ub}$ within the error in accuracy δ_{QPE}. The output and the ideal case thus give,

$$\begin{aligned} |\tilde{\phi}_E(\mathbf{y}_s) - \tilde{\phi}(\mathbf{y}_s)| &\leq \delta_{\text{QPE}} \\ C(\mathbf{y}_s) - C_{ub} \cdot \delta_{\text{QPE}} + \leq C_E(\mathbf{y}_s) &\leq C(\mathbf{y}_s) + C_{ub} \cdot \delta_{\text{QPE}}. \end{aligned} \tag{28}$$

This provides bounds to the output cost function value $C_E(\mathbf{y}_s)$, which is the same as the ideal cost function $C(\mathbf{y}_s)$ if $\delta_{QPE} = 0$.

Let $\mathbf{y}_s^* = \arg\max_{\mathbf{y}_s \in \mathcal{S}} C(\mathbf{y}_s)$, with the cost function $C(\mathbf{y}_s^*)$, being the optimal solution. Considering the amplitudes are uniform over the feasible region, i.e., $|F_n|^2 = 1/N_{\mathbf{y}_s}$. The success probability p for the solution in the ideal case ($\delta_{QPE} = 0$) is,

$$p = \frac{1}{N_{\mathbf{y}_s}} \cdot \left(1 - \frac{1}{(1 + C(\mathbf{y}_s^*))^2}\right). \tag{29}$$

and the post-selection success probabilities with non-zero δ_{QPE} are bounded as:

$$\begin{aligned} &\frac{1}{N_{\mathbf{y}_s}} \cdot \left(1 - \frac{1}{(1 + C(\mathbf{y}_s^*) - C_{ub} \cdot \delta_{\text{QPE}})^2}\right) \\ &\quad \leq p \leq \frac{1}{N_{\mathbf{y}_s}} \cdot \left(1 - \frac{1}{(1 + C(\mathbf{y}_s^*) + C_{ub} \cdot \delta_{\text{QPE}})^2}\right). \end{aligned} \tag{30}$$

The bounds on the probability p grow larger as δ_{QPE} is increased, leading to further uncertainty in the final solution.

Steps		Resource complexity		Time complexity T_Q
	Sub-steps	Qubit gates (single + multi)	Qudit gates (single + multi)	
1.	$\|\psi_1\rangle \mapsto \|\psi_2\rangle$	0	$n + 0$	$O(n)$
2.	$\|\psi_2\rangle \mapsto \|\psi_3\rangle$	0	$0 + O(mn^2)$	$O(m \cdot n^2 \cdot \log d \cdot \log \epsilon_u^{-1})$
3.	$\|\psi_3\rangle \mapsto \|\psi_4\rangle$	0	$\sim 6 \cdot d^{n/2} + d^{n/2}$	$O(d^{n/2} \cdot \log \epsilon_G^{-1})$
4.	$\|\psi_4\rangle \mapsto \|\psi_5\rangle$ $\|\psi_5\rangle \mapsto \|\psi_6\rangle$	0 $l + 0$	0 0	0 $O(l)$
5.	$\|\psi_6\rangle \mapsto \|\psi_7\rangle$ $\|\psi_7\rangle \mapsto \|\psi_8\rangle$	$0 + l$ $l + l$	$0 + O(n)$ 0	$O(n \cdot \epsilon_{QPE}^{-1})$ $O(l^2)$
6.	$\|\psi_8\rangle \mapsto \|\psi_9\rangle$	$0 + 1$	0	$O(\log \epsilon_R^{-1})$
7.	$\|\psi_9\rangle \mapsto \|\psi_{10}\rangle$	0	0	0
	Total	$(\sim 2l) + (2l + 1)$	$n + O(n + mn^2) + 6 \cdot d^{n/2} + d^{n/2}$	$O(n + m \cdot n^2 \cdot \log d \cdot \log \epsilon_u^{-1} + d^{n/2} \cdot \log \epsilon_G^{-1} + l + l^2 + n \cdot \epsilon_{QPE}^{-1} + \log \epsilon_R^{-1})$, in leading order $\approx O(m \cdot n^2 \cdot \log d + d^{n/2} + n \cdot \epsilon_{QPE}^{-1})$

Table I: Step-wise resource and time complexity analysis of the quantum algorithm. Each transformation $|\psi_i\rangle \mapsto |\psi_{i+1}\rangle$ corresponds to a specific subroutine for which single- and multi-qubit/qudit gates, and the associated computational time are calculated. Resource complexity is given in terms of the gate types, while time complexity is expressed as a function of qudits (n), qubits (m, l), qudit dimension d, and precision parameters ϵ_u, ϵ_G, ϵ_{QPE}, and ϵ_R. The last row indicates the total resource counts and provides an asymptotic time complexity dominated by leading-order terms.

Let $\mathbf{y}_s^\dagger \in \mathcal{S}$ be a near-optimal solution with cost function $C(\mathbf{y}_s^\dagger)$. Then, to resolve the probability between the optimal $\mathbf{y}_s^*$ and the sub-optimal solution $\mathbf{y}_s^\dagger$, the difference between the QPE estimates has to be more than the precision ϵ_{QPE}, leading to the condition,

$$\tilde{\phi}_E(\mathbf{y}_s^*) - \tilde{\phi}_E(\mathbf{y}_s^\dagger) = \frac{C_E(\mathbf{y}_s^*) - C_E(\mathbf{y}_s^\dagger)}{C_{ub}} > \epsilon_{\text{QPE}}, \tag{31}$$

when taking the lower limit of $C_E(\mathbf{y}_s^*)$ and upper limit of $C_E(\mathbf{y}_s^\dagger)$ gives the following condition,

$$C_E(\mathbf{y}_s^*) - C(\mathbf{y}_s^\dagger) > \epsilon_{\text{QPE}} \cdot C_{ub} + 2C_{ub} \cdot \delta_{\text{QPE}}. \tag{32}$$

This condition ensures that the probabilities corresponding to the optimal and the near-optimal solutions can be resolved. If the precision of QPE ϵ_{QPE} attains larger values, this condition will not be satisfied at a certain threshold, and hence, the probabilities will show degeneracies. In general, ϵ_{QPE} is given by the number of noiseless qubits l in the QPE as $1/2^l$, which depends on the experimental platform.

Number of Repetitions. The quantum algorithm outputs the optimal solution with probability p, given by Eq. 29. To increase the probability of finding the optimal solution, the algorithm needs to be repeated r times independently [53, 85]. The probability of failure (no success) in all r runs is

$$(1 - p)^r.$$

Hence, the probability of obtaining at least one success in r repetitions is

$$P(\text{at least one success}) = 1 - (1 - p)^r.$$

In order to guarantee this probability to be at least a target threshold P, then

$$\begin{aligned} & 1 - (1 - p)^r \geq P \\ \implies & (1 - p)^r \leq 1 - P, \\ \implies & r \cdot \log(1 - p) \leq \log(1 - P). \end{aligned} \tag{33}$$

Since $\log(1 - p) < 0$, rearranging yields

$$r \geq \frac{\log(1 - P)}{\log(1 - p)}. \tag{34}$$

Fig 5 shows the number of repetitions required for a given success probability to reach a target success probability of 0.51, 0.67, 0.80, and 0.999. All of the target probabilities are such that the algorithm provides the optimal solution more than 50% of the time, and shows that with $50 - 100$ repetitions, many of the problems (with $p \leq 0.1$) can be solved with high probability.

For small p or when P is close to 1, using the approximation $\log(1 - x) \approx -x$ for small x,

$$r \approx \frac{-\log(1 - P)}{p}. \tag{35}$$

For example, for $P = 0.99$, $-\log(1 - 0.99) = -\log(0.01) \approx 4.605$. And so the number of repetitions required and using Eq. 29 is

$$r \approx \frac{4.605}{p} = \frac{4.605 \cdot N_{\mathbf{y}_s}}{\left(1 - \frac{1}{(1+C(\mathbf{y}_s^*))^2}\right)}. \tag{36}$$

The number of repetitions to get to the optimal solution depends on the problem, which can be estimated using classical pre-processing of the problem. Using the branch and bound algorithm [86] can provide a concrete lower bound on the cost function value, which can bound the number of repetitions required. As the classical algorithm reports the first feasible solution, using that as the lower bound for $C(\mathbf{y}_s^*)$ leads to the maximum repetition r for a particular problem that guarantees the solution using the quantum algorithm.

V. DISCUSSION AND CONCLUSIONS

Integer Programming (IP) is an optimization framework with widespread applications in logistics, finance, manufacturing, and decision-making under constraints [61]. Despite

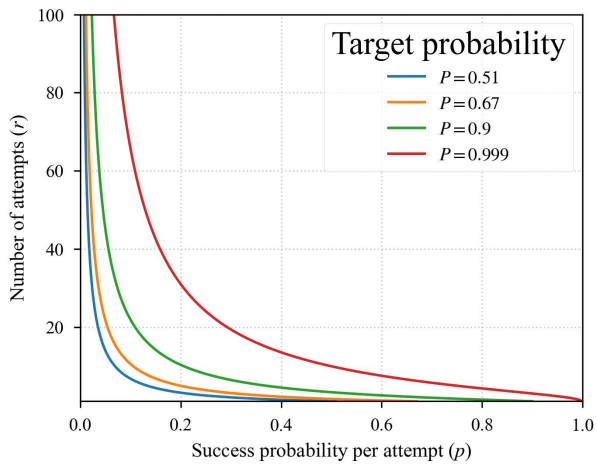


Figure 5: The relationship between the number of attempts (r) and the required success probability per attempt (p) to achieve various target probabilities (P). The curves show that as the success probability per attempt decreases, the required number of attempts increases. For a given target probability P, the success probability p is $p = 1 - (1 - P)^{1/r}$. The plot shows that more attempts allow for lower individual success probabilities while still achieving the same overall target probability.

the sophisticated classical algorithms, IP remains computationally intractable for large problem sizes due to its NP-hard nature [29]. This motivates the development of quantum algorithms that can provide a quantum advantage over classical methods, either through reduced complexity, improved scalability, or both. Most existing quantum approaches for combinatorial optimization problems, such as IP, rely on qubits or variational methods, which are often expensive in terms of resources. The lack of efficient encodings for integer variables has been a bottleneck in applying quantum algorithms to IP.

Building on prior work that solved small-scale IP using a single qudit [1], this work extends the algorithm to arbitrary IP problems by introducing a complete quantum circuit that achieves both constraint satisfaction and cost-function optimization. The measurement at the end yields the maximum probability for the optimal solution(s) by construction, leading to fewer circuit runs to achieve a high success probability as compared to a randomized algorithm. Our approach offers a provable exponential reduction in time complexity, from $T_c \sim O(d^n)$ for the classical case to

$$T_Q \sim O\left(d^{n/2} + m \cdot n^2 \cdot \log d + \frac{n}{\epsilon_{\mathrm{QPE}}}\right),$$

where n is the number of variables, each taking d values, m the number of constraints, and ϵ_{QPE} the QPE precision. This constitutes the best-known scaling for general IP, which constitutes the non-convex non-linear cases as well, and sets a new benchmark for quantum optimization algorithms. The subroutines in our algorithm are all performed using only quantum operations, which include the optimization of the classical cost function, hence shielding itself from the issues faced by variational quantum algorithms [87, 88]. Due to the measurement-based approach in the quantum algorithm, for practical purposes, this can distinguish between undecidable and decidable problem instances before performing the cost function optimization. The qubit register stores the information about the constraint satisfiability and the number of satisfied constraints, which can help to modify the problem by removing one or more constraints to make the problem decidable. All operations are implemented within a gate-based framework, providing a path toward fault tolerance, thereby motivating the experiments [89].

For the practical realization of our algorithm, the essential components required are qudits, qudit/qubit gates, and hybrid qubit-qudit entangling gates. The qudits can be realised in a quantum system with a large number of controllable degrees of freedom, for example hyperfine/electronic states in atoms [90, 91] or non-trivial superposition of them like Rydberg-dressed states [92, 93], frequency modes in photonic system [94, 95], rotational states in ultracold polar molecules [96–98] and the simulation of *synthetic* dimensions [99–101]. Furthermore, the field for experimental demostration of qudit gates is also an active area, to list a few, a qutrit ($d = 3$) CZ gate on superconducting qubits [102], a native 2-qudit entangling gate up to $d = 5$ dimensions on trapped ions [103], a 2-ququarts ($d = 4$) cross-resonance entangling gate using transmon-based architecture [104], and a qudit C-NOT gate for $d = 4$ on NV center platform [105]. Formulation for universal qudit gate sets (Pauli, Hadamard, Z-rotation, controlled, and SWAP gates) for implementing QFT [26] is also presented, along with a proposal for hybrid gates acting on qudits of different dimensions (2-qudit SUM, SWAP, many-qudit hybrid Toffoli and Fredkin gates) on spin systems [106]. However, for proof of principle of our algorithm, the near-term devices can solve a binary non-linear IP with the current gate sets available, which, for a highly non-convex problem, can potentially indicate an advantage over the classical algorithms.

As with all current quantum algorithms, there can be various sources of error. One of the significant issues that can arise is due to the errors in the unitary operator representing the distillation function that implements the constraints. This can cause states in the qudit register to leak out of the feasible region, leading to states from the infeasible region interfering with the optimization procedure. Despite this, the optimal solution remains measurable with a greater number of iterations. Once the integer values are decoded, the satisfaction of the constraints can be checked in polynomial time, and the next probable integer assignment can be chosen as the solution. The other sources of errors in the circuit, such as during the amplitude amplification and the QPE, are well studied with statements about their robustness [107–113]. Further study on the effect of experimental noise in our algorithm is an outlook for us. Also, hybrid algorithms combining quantum operations with classical post-processing could further improve the efficiency of the algorithm. Other IP problems containing trigonometric functions, logarithms, and rational polynomials ($P(X)/Q(X)$) in the cost function require further studies to be tackled within our framework.

ACKNOWLEDGMENTS

This work is funded by the German Federal Ministry of Education and Research within the funding program "Quantum Technologies - from basic research to market" under Contract No. 13N16138.

Appendix I: Generalization of the Hadamard gates for d-dimensional qudits

The Hadamard gate for a qudit with d dimensions is defined as

$$\hat{H}_d \left|\alpha\right\rangle = \frac{1}{\sqrt{d}} \sum_{\beta=0}^{d-1} (e^{2\pi \mathrm{i}/d})^{\alpha\beta} \left|\beta\right\rangle, \alpha \in \{0, 1, ..., d-1\}, \tag{I-1}$$

for a state $\left|\alpha\right\rangle = \left|\mathbf{0}\right\rangle$, it gives,

$$\hat{H}_d \left|\mathbf{0}\right\rangle = \frac{1}{\sqrt{d}} \sum_{\beta=0}^{d-1} \left|\beta\right\rangle .$$

n such Hadamard gates are used for the step $\left|\psi_1\right\rangle \rightarrow \left|\psi_2\right\rangle$ as given by Eq. 4 in the main text.

Appendix II: Devising the distillation function and its operator implementation

To define the distillation function f_i for a constraint C_i acting on a subset of the integer variables y^i. f_i has to follow the condition that it separates the states forming the feasible and the infeasible region. The required operation to be performed is,

$$\begin{aligned} \left|\mathbf{y}, 0\right\rangle^i \rightarrow \hat{U}_{f_i} \left|\mathbf{y}, 0\right\rangle^i &= \left|\mathbf{y}, 0 \oplus_2 f_i(y^i)\right\rangle^i \\ &\Longrightarrow \begin{cases} \left|\mathbf{y}, 0\right\rangle_k^i & : \text{Infeasible region} \\ \left|\mathbf{y}, 1\right\rangle_s^i & : \text{Feasible region} \end{cases} \\ &= \left|\mathbf{y}, 0\right\rangle_k^i + \left|\mathbf{y}, 1\right\rangle_s^i , \end{aligned} \tag{II-1}$$

where the operator $\hat{U}_{f_i}$ performs a boolean addition $\oplus_2$ of the distillation function f_i to the qubit register. The function f_i depends on the constraint $C_i < h_i$ is given by Eq. 6 in the main text, defined as

$$f_i(C_i, h_i) = 2^{\lfloor \frac{C_i}{h_i} \rfloor} \tag{II-2}$$

The function is such that, if the constraint is satisfied, the value of f_i is 1; otherwise, it's an even integer as shown below.

$$\begin{aligned} &\text{If} \begin{cases} \frac{C_i}{h_i} \geq 1 & : \text{Infeasible region} \\ \frac{C_i}{h_i} < 1 & : \text{Feasible region} \end{cases} \\ &\text{then} \begin{cases} \lfloor \frac{C_i}{h_i} \rfloor > 0 \in \mathbf{Z}^+ & : \text{Infeasible region} \\ \lfloor \frac{C_i}{h_i} \rfloor = 0 & : \text{Feasible region} \end{cases} \\ &\Longrightarrow f_i = \begin{cases} 2^{\lfloor \frac{C_i}{h_i} \rfloor} = \text{even } \mathbf{Z}^+ & : \text{Infeasible region} \\ 2^{\lfloor \frac{C_i}{h_i} \rfloor} = 1 & : \text{Feasible region.} \end{cases} \end{aligned} \tag{II-3}$$

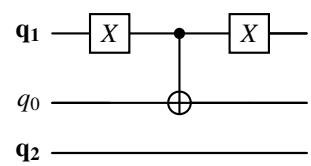
From Eq. II-1 along with the definition of f_i in Eq. II-3, the unitary $\hat{U}_{f_i}$ can be constructed, which is given by Eq. 7. For illustration of the matrix and the circuit for a $\hat{U}_{f_i}$ consider a simple constraint $C_1 : x_1 + 2x_2 < 2$, where $x_i \in \{0, 1\}$. The corresponding $\hat{U}_{f_1}$ implementing C_1 is a 8×8 matrix that specifically maps the following states,

$$\begin{aligned} \left|\mathbf{00}0\right\rangle &\xleftrightarrow{\text{Qubit flip}} \left|\mathbf{00}1\right\rangle \\ \left|\mathbf{01}0\right\rangle &\xleftrightarrow{\text{Qubit unchange}} \left|\mathbf{01}0\right\rangle \\ \left|\mathbf{10}0\right\rangle &\xleftrightarrow{\text{Qubit flip}} \left|\mathbf{10}1\right\rangle \\ \left|\mathbf{11}0\right\rangle &\xleftrightarrow{\text{Qubit unchange}} \left|\mathbf{11}0\right\rangle, \end{aligned}$$

and the corresponding $\hat{U}_{f_1}$ is,

$$\hat{U}_{f_1} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

which is a 1-sparse matrix. The corresponding circuit implementation using a $C - NOT$ gate is,



which flips q_0 only when $q_1 = 0$, requiring 3 gates.

Alternatively, the method involves constructing an 8×8 identity matrix from $\hat{U}_{f_i}$ by swapping rows/columns to reflect the transpositions, that is, a SWAP operation on rows $1 \leftrightarrow 2$ and rows $5 \leftrightarrow 6$.

Appendix III: Sequential application of m-unitary operators

The resulting state after sequential operations of m-$\hat{U}_{f_i}$ is,

$$\begin{aligned} \left|\psi_3\right\rangle &= \prod_{i=1}^{m} \hat{U}_{f_i} \left|\psi_2\right\rangle \\ &= \frac{1}{d^{n/2}} \prod_{i=1}^{m} \hat{U}_{f_i} \left(\sum_{\mathbf{y^i}=0}^{d^n-1} \left|\mathbf{y}\right\rangle^i \left|0\right\rangle^{\otimes m} \right) \\ &= \frac{1}{d^{n/2}} \left(\sum_{\mathbf{y}_{k_0}} \left|\mathbf{y}\right\rangle_{k_0} \left|00...0\right\rangle + \sum_{\mathbf{y}_{k_1}} \left|\mathbf{y}\right\rangle_{k_1} (\left|00...1\right\rangle + ...) \right. \\ &\left. + ... + \sum_{\mathbf{y}_{k_{m-1}}} \left|\mathbf{y}\right\rangle_{k_{m-1}} (\left|11...0\right\rangle + ...) + \sum_{\mathbf{y}_s \neq \mathbf{y}_{k_\gamma}} \left|\mathbf{y}\right\rangle_s \left|1\right\rangle^{\otimes m} \right) \\ &= \frac{1}{d^{n/2}} \left(\sum_{(k_\gamma, \gamma)} \left|\mathbf{y}\right\rangle_{k_\gamma} \left|q\right\rangle_\gamma + \sum_{\mathbf{y}_s \neq \mathbf{y}_{k_\gamma}} \left|\mathbf{y}\right\rangle_s \left|1\right\rangle^{\otimes m} \right) \end{aligned} \tag{III-1}$$

where $|q\rangle_\gamma \neq |1\rangle^{\otimes m}$ represents m qubit superposition state each with $\gamma \in [0, m-1]$ qubits in state $|1\rangle$, $|\mathbf{y}\rangle_{k_0}, ..., |\mathbf{y}\rangle_{k_\gamma}, ..., |\mathbf{y}\rangle_{k_{m-1}}$ are the corresponding qudit-states with $0, ..., (m-1)$ constraints satisfied respectively, and $|\mathbf{y}\rangle_s$ is the state which satisfies all the m constraints.

Appendix IV: Grover amplitude amplification

The Grover operator is defined as $\hat{G} = \hat{D}\hat{O}$, where $\hat{O} = \hat{I} - 2|M\rangle\langle M|$ is the phase flip operator and $\hat{D} = 2|\psi\rangle\langle\psi| - \hat{I}$ is the diffusion operator that performs a reflection around the mean. The goal is to maximize the amplitude of a given state $|M\rangle$ out of a superposition state $|\psi\rangle$. Let's take an m-qubit case, where the state with all the qubits is state $|1\rangle$ is the target state. The operator $\hat{O}$ in this case is as: $\hat{O} = \hat{I} - 2|1\rangle^{\otimes m}\langle 1|^{\otimes m}$, where $|1\rangle^{\otimes m}$ is the target state we are searching for. This operation applies a phase flip to $|1\rangle^{\otimes m}$ while leaving other states unchanged:

$$\hat{O}|1\rangle^{\otimes m} = -|1\rangle^{\otimes m}, \quad \hat{O}|x\rangle = |x\rangle \text{ for } x \neq 11...1.$$

The diffusion operator $\hat{D}$ enhances the amplitude of the target state by reflecting all amplitudes around their mean. It is given by: $\hat{D} = 2|\psi\rangle\langle\psi| - \hat{I}$, where $|\psi\rangle$ is the superposition state from which the target state needs to be sampled.

$$|\psi\rangle = \frac{1}{\sqrt{N}}\sum_x |x\rangle.$$

Applying $\hat{D}$ to the target state and its perpendicular state results in,

$$\begin{aligned}\hat{D}|1\rangle^{\otimes m} &= \cos(2\theta)|1\rangle^{\otimes m} + \sin(2\theta)(|1\rangle^{\otimes m})^\perp \\ \hat{D}(|1\rangle^{\otimes m})^\perp &= \sin(2\theta)|1\rangle^{\otimes m} - \cos(2\theta)(|1\rangle^{\otimes m})^\perp,\end{aligned} \tag{IV-1}$$

where θ is given by $\sin^2(\theta) = \frac{1}{N}$ and $N = 2^m$ in the case where $|\psi\rangle$ is in the uniform superposition of all the possible qubit states. Since the Grover operator $\hat{G}$ is a rotation in a 2D subspace spanned by $|1\rangle^{\otimes m}$ and its orthogonal complement $(|1\rangle^{\otimes m})^\perp$, its eigenstates are:

$$|\psi_\pm\rangle = \frac{1}{\sqrt{2}}\left(|1\rangle^{\otimes m} \pm i(|1\rangle^{\otimes m})^\perp\right).$$

The corresponding eigenvalues of $\hat{G}$ are $\lambda_\pm = e^{\pm i\theta}$. Since the search space is large ($N = 2^m$), for small $P_{11...1} = \frac{1}{N}$, we approximate: $\theta \approx 2\sqrt{P_{11...1}} = \frac{2}{\sqrt{N}}$. Each application of $\hat{G}$ rotates the initial uniform state $|\psi\rangle$ toward $|1\rangle^{\otimes m}$ by an angle θ. The number of iterations required to maximize $P_{11...1}$ is:

$$p_{\text{opt}} = \frac{\pi}{4}\frac{1}{\theta} = \frac{\pi}{4}\sqrt{N}.$$

After p_{opt} iterations, the probability of measuring the target state approaches 1.

Now, we detail the stepwise application of the operator $\hat{O}_m$ to the state $|\psi_3\rangle$ in our algorithm, as

$$\begin{aligned}&\hat{O}_m|\psi_3\rangle|-\rangle \\ &= \frac{1}{d^{n/2}}\hat{O}_m\left(\sum_{(k_\gamma,\gamma)}|\mathbf{y}\rangle_{k_\gamma}|q\rangle_\gamma + \sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}|\mathbf{y}\rangle_s|1\rangle^{\otimes m}\right)|-\rangle \\ &= \frac{1}{d^{n/2}}\left(\sum_{(k_\gamma,\gamma)}|\mathbf{y}\rangle_{k_\gamma}\hat{X}_{MC}\Big(|q\rangle_\gamma|-\rangle\Big)\right. \\ &\qquad\left. + \sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}|\mathbf{y}\rangle_s\hat{X}_{MC}\Big(|1\rangle^{\otimes m}|-\rangle\Big)\right) \\ &= \frac{1}{d^{n/2}}\left(\sum_{(k_\gamma,\gamma)}|\mathbf{y}\rangle_{k_\gamma}\Big(|q\rangle_\gamma|-\rangle\Big)\right. \\ &\qquad\left. + \sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}|\mathbf{y}\rangle_s(-1)\Big(|1\rangle^{\otimes m}|-\rangle\Big)\right) \\ &= \frac{1}{d^{n/2}}\left(\sum_{(k_\gamma,\gamma)}|\mathbf{y}\rangle_{k_\gamma}|q\rangle_\gamma - \sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}|\mathbf{y}\rangle_s|1\rangle^{\otimes m}\right)|-\rangle.\end{aligned} \tag{IV-2}$$

If the ancilla qubit is in state $|0\rangle$ instead of $|-\rangle$, applying a multi-controlled Z ($\hat{Z}_{MC}$) with control over the qubit register will result in the same marked state. The $\hat{Z}_{MC}$ gate can be implemented as, $\hat{X}_{MC}(a_1, ..., a_m; a) \quad \hat{Z}(a) \quad \hat{X}_{MC}(a_1, ..., a_m; a)$, where a represents the ancilla qubit and $(a_1 a_2 \cdots a_m)$ is the qubit register.

Let

$$d^{-n/2}\sum_{(k_\gamma,\gamma)}|\mathbf{y}\rangle_{k_\gamma}|q\rangle_\gamma \equiv \alpha_k|\mathbf{K}, Q\rangle$$

$$d^{-n/2}\sum_{\mathbf{y}_s\neq\mathbf{y}_{k_\gamma}}|\mathbf{y}\rangle_s|1\rangle^{\otimes m} \equiv \beta_s\left|\mathbf{S}, 1^{\otimes m}\right\rangle,$$

then $|\psi_3\rangle = \alpha_k|\mathbf{K}, Q\rangle + \beta_s\left|\mathbf{S}, 1^{\otimes m}\right\rangle$. And the diffusion operator $\hat{D}_m$ reflecting the state around the average is defined as,

$$\begin{aligned}\hat{D}_m &= 2|\psi_3\rangle\langle\psi_3| - \hat{I}, \\ &= 2\Big(\alpha_k|\mathbf{K}, Q\rangle + \beta_s\left|\mathbf{S}, 1^{\otimes m}\right\rangle\Big)\Big(\alpha_k^*\langle\mathbf{K}, Q| + \beta_s^*\left\langle\mathbf{S}, 1^{\otimes m}\right|\Big) \\ &\qquad - \hat{I},\end{aligned} \tag{IV-3}$$

where $|\psi_3\rangle$ is the initial superposition state of the system before the application of the operator $\hat{O}_m$, α_k^* and β_s^* are the complex conjugates of α_k and β_s, respectively.

Appendix V: Encoding of the cost function as the phase of the state

A prerequisite for QPE is to encode the cost function $C(\mathbf{y}_s)$ for each multi-qudit state in the feasible region to the l-qubit register. To do this, the operator $\hat{q}$ is,

$$\hat{q}\left|\mathbf{x_j}\right\rangle = x_j\left|\mathbf{x_j}\right\rangle, \tag{V-1}$$

where x_j is the eigenvalue of a single qudit state $\left|\mathbf{x_j}\right\rangle$. This eigenvalue operator is used to construct a unitary operator

$\hat{O}_c$ for imprinting the cost as the phase of the corresponding many-body qudit state. For any cost function of IP, we use the properties listed below for the implementation of $\hat{O}_c$.

Consider two sub-spaces of a system A and B with arbitrary operators, $\hat{H}_A$, $\hat{H}_B$ and Identity operators $\hat{I}_A$, $\hat{I}_B$. Then, $e^{\hat{H}_A \otimes \hat{I}_B} = e^{\hat{H}_A} \otimes \hat{I}_B$, the proof is given below.

$$
\begin{aligned}
Proof: e^{\hat{H}_A \otimes \hat{I}_B} &= \sum_{i=0}^{\infty} \frac{\left(\hat{H}_A \otimes \hat{I}_B\right)^i}{i!} \\
&= \left(\hat{I}_A \otimes \hat{I}_B\right) + \left(\hat{H}_A \otimes \hat{I}_B\right) + \frac{1}{2}\left(\hat{H}_A \otimes \hat{I}_B\right)^2 + ... \\
&= \left(\hat{I}_A \otimes \hat{I}_B\right) + \left(\hat{H}_A \otimes \hat{I}_B\right) + \frac{1}{2}\left(\hat{H}_A^2 \otimes \hat{I}_B^2\right) + ... \\
&= \left(\hat{I}_A + \hat{H}_A + \frac{1}{2}\hat{H}_A^2 + ...\right) \otimes \hat{I}_B \\
&= e^{\hat{H}_A} \otimes \hat{I}_B. \qquad \text{(V-2)}
\end{aligned}
$$

For a system with multiple sub-spaces A, B, C and D, it follows,

$$
e^{\hat{I}_A \otimes \hat{H}_B \otimes \hat{I}_C \otimes \hat{I}_D} = \hat{I}_A \otimes e^{\hat{H}_B} \otimes \hat{I}_C \otimes \hat{I}_D. \qquad \text{(V-3)}
$$

Consider the following eigenvalue equations,

$$
\hat{H}_A \left|\psi_A\right\rangle = \epsilon_A \left|\psi_A\right\rangle \text{ and } \hat{H}_B \left|\psi_B\right\rangle = \epsilon_B \left|\psi_B\right\rangle
$$

$$
\text{then: } e^{H_A \otimes H_B} \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle = e^{\epsilon_A \epsilon_B} \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle. \qquad \text{(V-4)}
$$

And the proof is given below.

$$
\begin{aligned}
Proof: & e^{\hat{H}_A \otimes \hat{H}_B} \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle \\
&= \sum_{i=0}^{\infty} \frac{\left(\hat{H}_A \otimes \hat{H}_B\right)^i}{i!} \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle \\
&= \left(\left(\hat{I}_A \otimes \hat{I}_B\right) + \left(\hat{H}_A \otimes \hat{H}_B\right) \right. \\
&\qquad \left. + \frac{1}{2}\left(\hat{H}_A \otimes \hat{H}_B\right)^2 + ...\right) \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle \\
&= \left(\hat{I}_A \otimes \hat{I}_B\right) \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle + \left(\hat{H}_A \otimes \hat{H}_B\right) \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle \\
&\qquad + \frac{1}{2}\left(\hat{H}_A^2 \otimes \hat{H}_B^2\right) \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle + ... \\
&= \left(1 + \left(\epsilon_A \epsilon_B\right) + \frac{1}{2}\left(\epsilon_A \epsilon_B\right)^2 + ...\right) \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle \\
&= e^{\epsilon_A \epsilon_B} \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle. \qquad \text{(V-5)}
\end{aligned}
$$

For a system with multiple sub-spaces,

$$
\begin{aligned}
& e^{\hat{H}_A \otimes \hat{H}_B \otimes \hat{H}_C \otimes \hat{I}_D} \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle \otimes \left|\psi_C\right\rangle \otimes \left|\psi_D\right\rangle \\
&\quad = e^{\epsilon_A \epsilon_B \epsilon_C} \left|\psi_A\right\rangle \otimes \left|\psi_B\right\rangle \otimes \left|\psi_C\right\rangle \otimes \left|\psi_D\right\rangle. \qquad \text{(V-6)}
\end{aligned}
$$

The encoding of the non-linear cost function using the above equations is given in the main text.

Step-wise encoding of a linear cost function as the phase of the state: Take a linear cost function of the form $C(\mathbf{y}_s) = a_1 x_1 + \cdots + a_n x_n$. The eigenvalue operator given by Eq. V-1 is used to construct a unitary operator $\hat{O}_c$ for imprinting the cost as the phase of the corresponding many-body qudit state,

$$
\hat{O}_c = e^{i2\pi \hat{q} a_1 / C_{ub}} \otimes \cdots \otimes e^{i2\pi \hat{q} a_n / C_{ub}}. \qquad \text{(V-7)}
$$

The action of the operator $\hat{O}_c$ for the exemplar linear cost function for the state $\left|\psi_6\right\rangle$ is,

$$
\begin{aligned}
\left|\tilde{\psi}_6\right\rangle &= \hat{O}_c \left|\psi_6\right\rangle \\
&= \hat{O}_c \sum_i \left(\tilde{F}_n \sum_{k=0}^{2^l-1} \left|k\right\rangle\right) \left(\sum_{\mathbf{y}_s \neq \mathbf{y}_{k_\gamma}} \left|\mathbf{y}\right\rangle_s\right) \left|0\right\rangle \\
&= \tilde{F}_n \left(e^{i2\pi \hat{q} a_1 / C_{ub}} \otimes \cdots \otimes e^{i2\pi \hat{q} a_n / C_{ub}}\right) \\
&\qquad \left(\sum_{\mathbf{y}_s \neq \mathbf{y}_{k_\gamma}} \left|\mathbf{x_1}\right\rangle_s \otimes \cdots \otimes \left|\mathbf{x_n}\right\rangle_s\right) \otimes \sum_i \left(\sum_{k=0}^{2^l-1} \left|k\right\rangle\right) \left|0\right\rangle \\
&= \tilde{F}_n \left(\sum_{\mathbf{y}_s \neq \mathbf{y}_{k_\gamma}} e^{i2\pi \hat{q} a_1 / C_{ub}} \left|\mathbf{x_1}\right\rangle_s \otimes \cdots \right. \\
&\qquad \left. \otimes e^{i2\pi \hat{q} a_n / C_{ub}} \left|\mathbf{x_n}\right\rangle_s\right) \otimes \sum_i \left(\sum_{k=0}^{2^l-1} \left|k\right\rangle\right) \left|0\right\rangle \\
&= \tilde{F}_n \left(\sum_{\mathbf{y}_s \neq \mathbf{y}_{k_\gamma}} e^{i2\pi (a_1 x_1^s + ... + a_n x_n^s) / C_{ub}} \right. \\
&\qquad \left. \left(\left|\mathbf{x_1}\right\rangle_s \otimes \cdots \otimes \left|\mathbf{x_n}\right\rangle_s\right)\right) \otimes \sum_i \left(\sum_{k=0}^{2^l-1} \left|k\right\rangle\right) \left|0\right\rangle \\
&= \tilde{F}_n \left(\sum_{\mathbf{y}_s \neq \mathbf{y}_{k_\gamma}} e^{i2\pi C(\mathbf{y}_s) / C_{ub}} \left|\mathbf{y}\right\rangle_s\right) \otimes \sum_i \left(\sum_{k=0}^{2^l-1} \left|k\right\rangle\right) \left|0\right\rangle, \qquad \text{(V-8)}
\end{aligned}
$$

where $\left|\mathbf{x_i}\right\rangle_s$ are the individual integer states of the qudit i corresponding to a particular superposition state represented by $\left|\mathbf{y}\right\rangle_s$.

Appendix VI: Determining the convexity for a non-linear IP

The convexity of the nonlinear sample problem given by Eq. 26 is analyzed by examining its Hessian matrices and eigenvalues. The convexity of an IP problem is defined by the properties of the surface spanned by the constraints. If the eigenvalues of the Hessian corresponding to a constraint are positive, then it is a convex inequality constraint; otherwise, it is non-convex, which refers to the optimization problem being non-convex.

Constraint 1: $C_1(\boldsymbol{x}) = x_1 + x_2^2 x_3 + x_3 - 1 < 0$, with the non-linear part:

$$
h_1(x_2, x_3) = x_2^2 x_3 + x_3.
$$

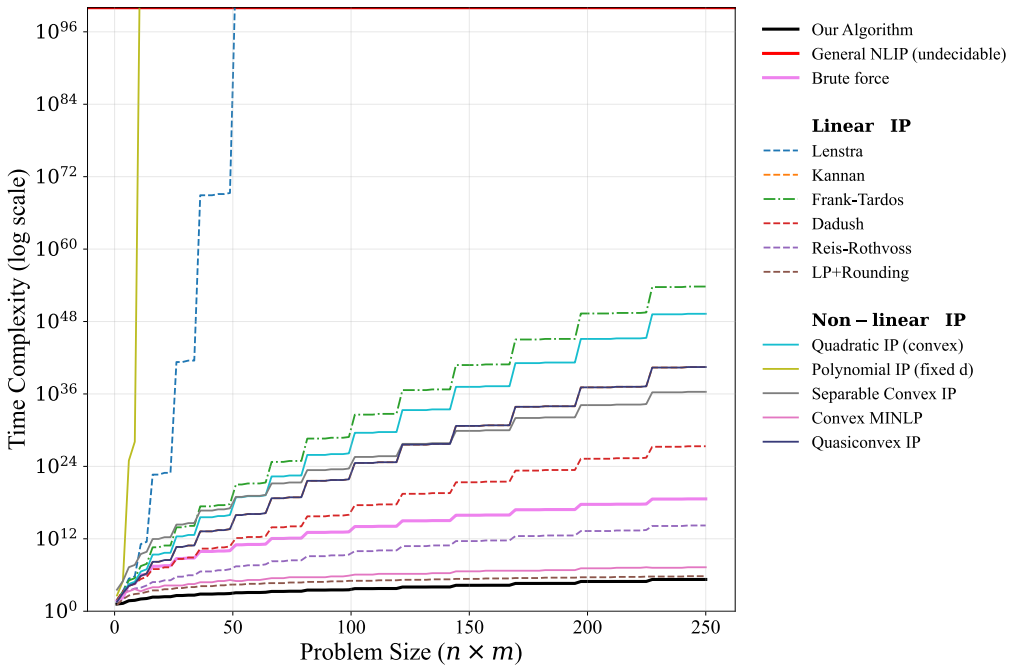


Figure A1: Comparative analysis of integer programming algorithms. Time complexity scaling with problem size ($n \times m$) with $d = 5$, $V = 10$, $\Delta = 100$, $k = 3$, $c = 2$ and $\epsilon_{QPE} = 0.1$.

Partial derivatives of h_1 is calculated below,

$$\frac{\partial^2 h_1}{\partial x_2^2} = 2x_3, \quad \frac{\partial^2 h_1}{\partial x_2 \partial x_3} = 2x_2, \quad \frac{\partial^2 h_1}{\partial x_3^2} = 0,$$

to give the Hessian matrix as,

$$H(h_1) = \begin{bmatrix} 2x_3 & 2x_2 \\ 2x_2 & 0 \end{bmatrix},$$

and the eigenvalues of $H(h_1)$ are given by:

$$\lambda_1 = x_3 \pm \sqrt{x_3^2 + 4x_2^2},$$

where $\sqrt{x_3^2 + 4x_2^2} \geq |x_3|$, leading to one eigenvalue being always positive and the other one satisfying

$$\lambda_{1-} = x_3 - \sqrt{x_3^2 + 4x_2^2} \leq 0.$$

Thus, H_1 is indefinite and h_1 is non-convex.

Similarly for other constraints:

$$C_2(\boldsymbol{x}) = 3x_3^2 x_4 + x_2 - 2 < 0$$
$$h_2(x_3, x_4) = 3x_3^2 x_4$$

$$C_3(\boldsymbol{x}) = x_1 x_5 + x_4 - 1 < 0$$
$$h_3(x_1, x_5) = x_1 x_5$$

$$C_4(\boldsymbol{x}) = 2x_1 + 2x_1^2 x_3 + x_4^3 - 2 < 0$$
$$h_4(x_1, x_3, x_4) = 2x_1^2 x_3 + x_4^3. \qquad \text{(VI-1)}$$

The corresponding Hessian matrices are:

$$H(h_2) = \begin{bmatrix} 6x_4 & 6x_3 \\ 6x_3 & 0 \end{bmatrix}, \quad H(h_3) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$
$$H(h_4) = \begin{bmatrix} 4x_3 & 4x_1 & 0 \\ 4x_1 & 0 & 0 \\ 0 & 0 & 6x_4 \end{bmatrix}. \qquad \text{(VI-2)}$$

and the eigenvalues are:

$$\lambda_2 = 3x_4 \pm 3\sqrt{x_4^2 + 4x_3^2}, \quad \lambda_3 = \pm 1,$$
$$\lambda_4 = 2x_3 \pm 2\sqrt{x_3^2 + 4x_1^2}$$
$$\text{for the block containing } x_1, x_3 \text{ in } H(h_4). \qquad \text{(VI-3)}$$

Each nonlinear constraint function h_i has a Hessian matrix with at least one negative eigenvalue (is indefinite), hence, all the constraints are **non-convex**.

The Fig. A1 compares the classical algorithms for linear and non-linear IP with the quantum algorithm for the parameters described in the caption. The algorithms solving non-linear problems apply to convex IP.

[1] Kapil Goswami, Peter Schmelcher, and Rick Mukherjee. Integer programming using a single atom. *Quantum Sci. Technol.*, 9(4):045016, 2024.

[2] Alexander M Dalzell, Sam McArdle, Mario Berta, Przemyslaw Bienias, Chi-Fang Chen, András Gilyén, Connor T Hann, Michael J Kastoryano, Emil T Khabiboulline, Aleksander Kubica, et al. Quantum algorithms: A survey of applications and end-to-end complexities. *arXiv:2310.03011*, 2023.

[3] Ashley Montanaro. Quantum algorithms: an overview. *NPJ Quantum Inf.*, 2(1):1, 2016.

[4] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, et al. Variational quantum algorithms. *Nat. Rev. Phys.*, 3(9):625, 2021.

[5] Peter W Shor. Introduction to quantum algorithms. *Proc. Symp. Appl. Math.*, 58:143, 2002.

[6] Pawan Kumar Mandal. A review of classical methods and nature-inspired algorithms (nias) for optimization problems. *Results Control Optim.*, 13:100315, 2023.

[7] Amira Abbas, Andris Ambainis, Brandon Augustino, Andreas Bärtschi, Harry Buhrman, Carleton Coffrin, Giorgio Cortiana, Vedran Dunjko, Daniel J Egger, Bruce G Elmegreen, et al. Challenges and opportunities in quantum optimization. *Nat. Rev. Phys.*, 6:718, 2024.

[8] Laurence A Wolsey. *Integer programming*, page 1. John Wiley & Sons, 2020.

[9] Alexander Schrijver. *Theory of linear and integer programming*, page 266. John Wiley & Sons, 1998.

[10] David Deutsch and Richard Jozsa. Rapid solution of problems by quantum computation. *Proc. R. Soc. Lond. A Math. Phys. Eng. Sci.*, 439(1907):553, 1992.

[11] David Deutsch. Quantum theory, the church–turing principle and the universal quantum computer. *Proc. R. Soc. Lond. A Math. Phys. Sci.*, 400(1818):97, 1985.

[12] Ethan Bernstein and Umesh Vazirani. Quantum complexity theory. *Proc. Annu. ACM Symp. Theory Comput.*, page 11, 1993.

[13] Koji Nagata, Han Geurdes, Santanu Kumar Patro, Shahrokh Heidari, Ahmed Farouk, and Tadao Nakamura. Generalization of the bernstein–vazirani algorithm beyond qubit systems. *Quantum Stud.: Math. Found.*, 7:17, 2020.

[14] Daniel R Simon. On the power of quantum computation. *SIAM J. Comput.*, 26(5):1474, 1997.

[15] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. *Proc. 35th Annu. Symp. Found. Comput. Sci.*, page 124, 1994.

[16] Don Coppersmith. An approximate fourier transform useful in quantum factoring. *arXiv:quant-ph/0201067*, 2002.

[17] A Yu Kitaev. Quantum measurements and the abelian stabilizer problem. *arXiv preprint quant-ph/9511026*, 1995.

[18] Lov K Grover. Quantum mechanics helps in searching for a needle in a haystack. *Phys. Rev. Lett.*, 79(2):325, 1997.

[19] Emanuel Knill. Quantum computing with realistically noisy devices. *Nature*, 434(7029):39, 2005.

[20] Yuchen Wang, Zixuan Hu, Barry C Sanders, and Sabre Kais. Qudits and high-dimensional quantum computing. *Front. Phys.*, 8:589504, 2020.

[21] Yulin Chi, Nina Richey, Nan Yao, and et al. A programmable qudit-based quantum processor. *Nat. Commun.*, 13(1):1166, 2022.

[22] Anastasiia S Nikolaeva, Evgeniy O Kiktenko, and Aleksey K Fedorov. Efficient realization of quantum algorithms with qudits. *EPJ Quantum Technol.*, 11(1):1, 2024.

[23] Yannick Deller, Matteo Brunelli, Andrew J. Daley, and et al. Quantum approximate optimization algorithm for qudit systems. *Phys. Rev. A*, 107(6):062410, 2023.

[24] Anastasiia S. Nikolaeva, Evgeniy O. Kiktenko, Aleksey K. Fedorov, and et al. Efficient realization of quantum algorithms with qudits. *EPJ Quantum Technol.*, 11(1):1, 2024.

[25] Byungjoo Kim, Peng Liu, Carlos Martinez, and et al. Qudit-based variational quantum eigensolver using photonic orbital angular momentum states. *Sci. Adv.*, 10(43):eado3472, 2024.

[26] Francesco Pudda, Mario Chizzini, Luca Crippa, and et al. Generalised quantum gates for qudits and their application in quantum fourier transform. *arXiv:2410.05122*, 2024.

[27] Richard M Karp et al. Complexity of computer computations. *Reducibility Among Combinatorial Problems*, 23(1):85, 1972.

[28] Ravindran Kannan and Clyde L Monma. On the computational complexity of integer programming problems. *Optim. Oper. Res.*, 157:161, 1978.

[29] Christos H Papadimitriou and Mihalis Yannakakis. The complexity of facets (and some facets of complexity). *J. Comput. Syst. Sci.*, 28:244, 1984.

[30] Charles H Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM J. Comput.*, 26(5):1510, 1997.

[31] Scott Aaronson, Greg Kuperberg, and Christopher Granade. The complexity zoo, 2005.

[32] Vangelis Th Paschos. An overview on polynomial approximation of np-hard problems. *Yugoslav J. Oper. Res.*, 19(1):3, 2009.

[33] Niklas Pirnay, Vincent Ulitzsch, Frederik Wilde, Jens Eisert, and Jean-Pierre Seifert. An in-principle super-polynomial quantum advantage for approximating combinatorial optimization problems via computational learning theory. *Sci. Adv.*, 10(11):eadj5170, 2024.

[34] Andrew Lucas. Ising formulations of many np problems. *Front. Phys.*, 2:5, 2014.

[35] Francisco Barahona, Martin Grötschel, Michael Jünger, and Gerhard Reinelt. An application of combinatorial optimization to statistical physics and circuit layout design. *Oper. Res.*, 36(3):493, 1988.

[36] Fred Glover, Gary Kochenberger, and Yu Du. A tutorial on formulating and using qubo models. *arXiv:1811.11538*, 2018.

[37] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv:1411.4028*, 2014.

[38] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O'brien. A variational eigenvalue solver on a photonic quantum processor. *Nat. Commun.*, 5(1):4213, 2014.

[39] Kapil Goswami, Rick Mukherjee, Herwig Ott, and Peter Schmelcher. Solving optimization problems with local light-shift encoding on rydberg quantum annealers. *Phys. Rev. Res.*, 6:023031, Apr 2024.

[40] Masayuki Ohzeki and Hidetoshi Nishimori. Quantum annealing: An introduction and new developments. *J. Comput. Theor. Nanosci.*, 8(6):963, 2011.

[41] Stuart Hadfield, Zhihui Wang, Bryan O'gorman, Eleanor G Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019.

[42] Yves Pochet and Laurence A Wolsey. *Production planning by mixed integer programming*, volume 149, page 163. Springer, 2006.

[43] Leandro Magatão, Lucia VR Arruda, and Flávio Neves Jr. A mixed integer programming approach for scheduling commodities in a pipeline. *Comput. Chem. Eng.*, 28(1), 2004.

[44] Chin-Yao Chang, Eric Jones, Yiyun Yao, Peter Graf, and Rishabh Jain. On hybrid quantum and classical computing algorithms for mixed-integer programming. *arXiv:2010.07852*, 2020.

[45] Shuntaro Okada, Masayuki Ohzeki, and Shinichiro Taguchi. Efficient partition of integer optimization problems with one-hot encoding. *Sci. Rep.*, 9(1):13036, 2019.

[46] Marika Svensson, Martin Andersson, Mattias Grönkvist, Pontus Vikstål, Devdatt Dubhashi, Giulia Ferrini, and Göran Johansson. Hybrid quantum-classical heuristic to solve large-scale integer linear programs. *Phys. Rev. Appl.*, 20(3):034062, 2023.

[47] Kapil Goswami, Gagan Anekonda Veereshi, Peter Schmelcher, and Rick Mukherjee. Solving the travelling salesman problem using a single qubit. *arXiv:2407.17207*, 2024.

[48] Daan Camps, Lin Lin, Roel Van Beeumen, and Chao Yang. Explicit quantum circuits for block encodings of certain sparse matrices. *SIAM J. Matrix Anal. Appl.*, 45(1):801, 2024.

[49] Igor Gaidai and Rebekah Herrman. Decomposition of sparse amplitude permutation gates with application to preparation of sparse clustered quantum states. *arXiv:2504.08705*, 2025.

[50] Lvzhou Li and Jingquan Luo. Nearly optimal circuit size for sparse quantum state preparation. *52nd International Colloquium on Automata, Languages, and Programming (ICALP 2025)*, page 113, 2025.

[51] Alvin Gonzales, Rebekah Herrman, Colin Campbell, Igor Gaidai, Ji Liu, Teague Tomesh, and Zain H Saleem. Arbitrary state preparation via quantum walks. *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2:616, 2024.

[52] Dominic W Berry, Andrew M Childs, Richard Cleve, Robin Kothari, and Rolando D Somma. Exponential improvement in precision for simulating sparse hamiltonians. *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, page 283, 2014.

[53] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *arXiv:quant-ph/0005055*, 2000.

[54] Lin Lin. Lecture notes on quantum algorithms for scientific computation. *arXiv:2201.08309*, 2022.

[55] Aram W Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Phys. Rev. Lett.*, 103(15):150502, 2009.

[56] Nikhil S Mande and Ronald de Wolf. Tight bounds for quantum phase estimation and related problems. *arXiv:2305.04908*, 2023.

[57] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge University Press, 2010.

[58] Yaakov S Weinstein, MA Pravia, EM Fortunato, Seth Lloyd, and David G Cory. Implementation of the quantum fourier transform. *Phys. Rev. Lett.*, 86(9):1889, 2001.

[59] My T Thai. Approximation algorithms: Lp relaxation, rounding, and randomized rounding techniques. *Lecture Notes, University of Florida*, 2013.

[60] Robert C Jeroslow. There cannot be any algorithm for integer programming with quadratic constraints. *Oper. Res.*, 21(1):221, 1973.

[61] Laurence A Wolsey and George L Nemhauser. *Integer and combinatorial optimization*. John Wiley & Sons, 1999.

[62] Margaret Wright. The interior-point revolution in optimization: history, recent developments, and lasting consequences. *Bull. Amer. Math. Soc.*, 42(1):39, 2005.

[63] The code can be obtained by contacting the authors.

[64] Hamed Mohammadbagherpoor, Young-Hyun Oh, Patrick Dreher, Anand Singh, Xianqing Yu, and Andy J Rindos. An improved implementation approach for quantum phase estimation on quantum computers. *2019 IEEE International Conference on Rebooting Computing (ICRC)*, page 1, 2019.

[65] Nick S Blunt, Laura Caune, Róbert Izsák, Earl T Campbell, and Nicole Holzmann. Statistical phase estimation and error mitigation on a superconducting quantum processor. *PRX Quantum*, 4(4):040341, 2023.

[66] Kentaro Yamamoto, Yuta Kikuchi, David Amaro, Ben Criger, Silas Dilkes, Ciarán Ryan-Anderson, Andrew Tranter, Joan M Dreiling, Dan Gresh, Cameron Foltz, et al. Quantum error-corrected computation of molecular energies. *arXiv:2505.09133*, 2025.

[67] Stefano Paesani, Andreas A Gentile, Raffaele Santagati, Jianwei Wang, Nathan Wiebe, David P Tew, Jeremy L O'Brien, and Mark G Thompson. Experimental bayesian quantum phase estimation on a silicon photonic chip. *Phys. Rev. Lett.*, 118(10):100503, 2017.

[68] Luca Pezzè and Augusto Smerzi. Quantum phase estimation algorithm with gaussian spin states. *PRX Quantum*, 2(4):040301, 2021.

[69] L Perron and F Didier. Or-tools cp-sat v9. 6, 2024.

[70] Hendrik W Lenstra Jr. Integer programming with a fixed number of variables. *Math. Oper. Res.*, 8(4):538, 1983.

[71] Ravi Kannan. Minkowski's convex body theorem and integer programming. *Math. Oper. Res.*, 12(3):415, 1987.

[72] András Frank and Éva Tardos. An application of simultaneous diophantine approximation in combinatorial optimization. *Combinatorica*, 7:49, 1987.

[73] Daniel Nicolas Dadush. *Integer programming, lattice algorithms, and deterministic volume estimation*. Georgia Institute of Technology, 2012.

[74] Victor Reis and Thomas Rothvoss. The subspace flatness conjecture and faster integer programming. *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, page 974, 2023.

[75] Friedrich Eisenbrand and Gennady Shmonin. Parametric integer programming in fixed dimension. *Math. Oper. Res.*, 33(4):839, 2008.

[76] Mohit Tawarmalani and Nikolaos V Sahinidis. *Convexification and global optimization in continuous and mixed-integer nonlinear programming: theory, algorithms, software, and applications*, volume 65. Springer Science & Business Media, 2013.

[77] Silvano Martello and Paolo Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, 1990.

[78] Mustafa R Kılınç and Nikolaos V Sahinidis. Exploiting integrality in the global optimization of mixed-integer nonlinear programming problems with baron. *Optim. Methods Softw.*, 33(3):540, 2018.

[79] Erling D Andersen and Knud D Andersen. The mosek interior point optimizer for linear programming: an implementation of the homogeneous algorithm. *High performance optimization*, page 197, 2000.

[80] Bernhard Meindl and Matthias Templ. Analysis of commer-

cial and free and open source solvers for linear optimization problems. *Eurostat and Statistics Netherlands within the project ESSnet on common tools and harmonised methodology for SDC in the ESS*, 20:64, 2012.

[81] Raymond Hemmecke, Matthias Köppe, Jon Lee, and Robert Weismantel. Nonlinear integer programming. *50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art*, page 561, 2009.

[82] See Min Lim, Cristian E Susa, and Ron Cohen. Curve-fitted qpe: Extending quantum phase estimation results for a higher precision using classical post-processing. *arXiv:2409.15752*, 2024.

[83] Sofia Martínez-Garaot, Marisa Pons, and Dmitri Sokolovski. From quantum probabilities to quantum amplitudes. *Entropy*, 22(12):1389, 2020.

[84] Joran van Apeldoorn. Quantum probability oracles & multidimensional amplitude estimation. *Proc. 16th Conf. Theory Quantum Comput., Commun. Cryptogr. (TQC 2021)*, page 9:1, 2021.

[85] Tomoki Tanaka, Shumpei Uno, Tamiya Onodera, Naoki Yamamoto, and Yohichi Suzuki. Noisy quantum amplitude estimation without noise estimation. *Phys. Rev. A*, 105(1):012411, 2022.

[86] EML Beale and RE Small. Mixed integer programming by a branch and bound technique. *Proceedings of the IFIP Congress*, 2:450, 1965.

[87] Lennart Bittel and Martin Kliesch. Training variational quantum algorithms is np-hard. *Phys. Rev. Lett.*, 127(12):120502, 2021.

[88] Lennart Bittel, Sevag Gharibian, and Martin Kliesch. Optimizing the depth of variational quantum algorithms is strongly qcma-hard to approximate. *arXiv:2211.12519*, 2022.

[89] Arkady K Fedorov, Nicolas Gisin, Serguei M Beloussov, and Alexander I Lvovsky. Quantum computing at the quantum advantage threshold: a down-to-business review. *arXiv:2203.17181*, 2022.

[90] SK Kanungo, JD Whalen, Y Lu, M Yuan, S Dasgupta, FB Dunning, KRA Hazzard, and TC Killian. Realizing topological edge states with rydberg-atom synthetic dimensions. *Nat. Commun.*, 13(1):972, 2022.

[91] Bryce Gadway. Atom-optics approach to studying transport phenomena. *Phys. Rev. A*, 92(4):043606, 2015.

[92] Tommaso Macri and Thomas Pohl. Rydberg dressing of atoms in optical lattices. *Phys. Rev. A*, 89(1):011402, 2014.

[93] Rick Mukherjee. Charge dynamics of a molecular ion immersed in a rydberg-dressed atomic lattice gas. *Phys. Rev. A*, 100(1):013403, 2019.

[94] Tomoki Ozawa and Hannah M Price. Topological quantum matter in synthetic dimensions. *Nat. Rev. Phys.*, 1(5):349, 2019.

[95] Luqi Yuan, Qian Lin, Meng Xiao, and Shanhui Fan. Synthetic dimension in photonics. *Optica*, 5(11):1396, 2018.

[96] JP Shaffer, ST Rittenhouse, and HR Sadeghpour. Ultracold rydberg molecules. *Nat. Commun.*, 9(1):1965, 2018.

[97] Rahul Sawant, Jacob A Blackmore, Philip D Gregory, Jordi Mur-Petit, Dieter Jaksch, Jesús Aldegunde, Jeremy M Hutson, Michael R Tarbutt, and Simon L Cornish. Ultracold polar molecules as qudits. *New J. Phys.*, 22(1):013027, 2020.

[98] Bryce Gadway and Bo Yan. Strongly interacting ultracold polar molecules. *J. Phys. B*, 49(15):152002, 2016.

[99] O Boada, A Celi, JI Latorre, and M Lewenstein. Quantum simulation of an extra dimension. *Phys. Rev. Lett.*, 108(13):133001, 2012.

[100] Ivar Martin, Gil Refael, and Bertrand Halperin. Topological frequency conversion in strongly driven quantum systems. *Phys. Rev. X*, 7(4):041008, 2017.

[101] Bhuvanesh Sundar, Bryce Gadway, and Kaden RA Hazzard. Synthetic dimensions in ultracold polar molecules. *Sci. Rep.*, 8(1):1, 2018.

[102] Noah Goss, Rebecca Smith, Arjun Patel, and et al. High-fidelity qutrit entangling gates for superconducting circuits. *Nat. Commun.*, 13(1):7481, 2022.

[103] Pavel Hrmo, Martin Niedermayr, Duncan Wilson, and et al. Native qudit entanglement in a trapped ion quantum processor. *Nat. Commun.*, 14(1):2242, 2023.

[104] Laurin E. Fischer, Christopher Eichler, Robert Versluis, and et al. Universal qudit gate synthesis for transmons. *PRX Quantum*, 4(3):030327, 2023.

[105] Fang-Fang Du, Jing Lin, Wei Zhao, and et al. Qudit-based high-dimensional controlled-not gate. *Opt. Lett.*, 49(5):1229, 2024.

[106] Jamil Daboul, Xiaoguang Wang, Barry C. Sanders, and et al. Quantum gates on hybrid qudits. *J. Phys. A: Math. Gen.*, 36(10):2525, 2003.

[107] Xiantao Li. Some error analysis for the quantum phase estimation algorithms. *J. Phys. A-Math. Theor.*, 55(32):325303, 2022.

[108] Adam M Meier, Karl A Burkhardt, Brian J McMahon, and Creston D Herold. Testing the robustness of robust phase estimation. *Phys. Rev. A*, 100(5):052106, 2019.

[109] Hiroo Azuma. Decoherence in grover's quantum algorithm: Perturbative approach. *Phys. Rev. A*, 65(4):042311, 2002.

[110] Pedro J Salas. Noise effect on grover algorithm. *Eur. Phys. J. D*, 46:365, 2008.

[111] Alexander A. Pomeransky, Oleg V. Zhirov, and Dima L. Shepelyansky. Effects of decoherence and imperfections for quantum algorithms. *arXiv:quant-ph/0407264*, 2004.

[112] OV Zhirov and DL Shepelyansky. Dissipative decoherence in the grover algorithm. *Eur. Phys. J. D*, 38:405, 2006.

[113] Daniel Shapira, Shay Mozes, and Ofer Biham. Effect of unitary noise on grover's quantum search algorithm. *Quantum Inf. Process.*, 3(6):331, 2004.